



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

DIPARTIMENTO DI MATEMATICA, INFORMATICA E
GEOSCIENZE

Corso di Laurea Triennale in Matematica

**Il teorema di Whitney-Graustein poligonale:
un approccio algoritmico computazionale**

Relatore:

Prof. Daniele Zuddas

Correlatore:

Prof. Matteo Gallet

Laureanda:

Ramani Isabella

ANNO ACCADEMICO 2024/2025

Ringraziamenti

Giunta al termine di questo percorso, desidero rivolgere il mio pensiero a chi mi ha aiutata a renderlo possibile.

Grazie al prof. Zuddas, per tutto il tempo che mi ha dedicato e la cura preziosa con cui svolge il suo lavoro.

Grazie al prof. Gallet, per la dedizione e l'empatia con cui trasmette la sua passione agli studenti.

Grazie ad Emy, mia sorella e insostituibile compagna di vita, per il tuo dolce e incondizionato sostegno e la condivisione delle gioie più grandi. Ti sono grata per essere stata la presenza costante di ogni mio ieri, e per la meravigliosa certezza che ci sarai in ogni mio domani.

Grazie a mamma, per la tua instancabile vivacità e per non avermi mai fatto mancare la purezza del tuo amore materno. Il tuo esempio è per me inestimabile.

Grazie a papà, per aver protetto il mio cammino e avermi sempre fatta sentire compresa. Per essere stato un'incrollabile certezza e una persona su cui so di poter contare.

Grazie ai nonni Renzo e Sandra e a nonna Fior, radici preziose della mia vita fin da bambina. Siete stati un porto sicuro in ogni istante, sempre pronti per una chiacchierata, per regalarmi un racconto di vita o ascoltarmi pazientemente e consigliarmi.

Grazie a Laura e zia, affetti familiari che non necessitano di consuetudine per preservarsi sinceri e profondi nel tempo.

Grazie a Ruggy, per avermi sempre teso la mano in ogni situazione dall'inizio alla fine del percorso e per l'affetto incondizionato che non bastano le parole per descrivere.

Grazie a Gioele, è stato bello crescere sapendo che il supporto e le risate più sincere, assieme ad un'invidiabile complicità, mi aspettavano appena fuori dalla porta di casa.

Grazie a Cecilia, per la condivisione dei traguardi universitari, ma anche di momenti di maggiore sconforto: ti sono grata per il continuo confronto reciproco che ci ha rese migliori nel tempo.

Grazie a Gaia, per aver camminato al mio fianco fin dai banchi del liceo, arricchendo il mio percorso con la tua sensibilità e con la generosità con cui da sempre ti dedichi agli altri.

Grazie a Cate, per l'inimitabile impegno con cui coltivi la nostra amicizia; grazie a Chiara, per la tua naturalezza e i momenti che abbiamo passato assieme dalle elementari. Grazie a entrambe per il dono di farmi sentire a casa ogni volta che siamo insieme.

Grazie a Simone, per l'esempio e il supporto che mi hai dato, tanto nella matematica quanto nell'amicizia.

Grazie a Giacomo, per la rara capacità con cui trasmetti le tue passioni e per aver condiviso con me il cuore che metti in ciò che fai e nei legami con le persone.

Grazie a Ilaria, compagna di gite e di infinite estati al mare, un legame che, posso dire con certezza, il tempo non cancellerà mai.

Grazie a Giulia, per la continua fonte d'ispirazione, ed un'amicizia che con sincerità ci spinge ogni giorno a migliorarci.

Grazie a Filippo e Lollo, i miei "angeli" matematici: grazie alle risate e all'aiuto reciproco in questi anni avete reso più belli teoremi e dimostrazioni.

Grazie alle Cussine: grazie ad Hana, la mia prima vera amica al cus; ad Ire, per le mille avventure condivise assieme, e a Fede per credere sempre in me. Grazie a Giu, Sivi, Mati, Marta, Annie, Leti, Vivi e Clo per aver reso la pallavolo una fonte inesauribile di felicità e un rifugio sicuro in questi anni.

Indice

Introduzione	iii
1 Breve storia del teorema di Whitney-Graustein	1
1.1 Il teorema	1
1.2 Poligonazione	3
1.3 Allisciamento	4
2 Preliminari	5
2.1 Poligonalità	5
2.2 Regolarità di poligonalità e numero di avvolgimento	6
2.3 Trasformazioni regolari	9
3 Poligonalità stellate	11
3.1 Lemmi tecnici	14
4 Il teorema di Whitney-Graustein poligonale	19
4.1 Il teorema	19
4.2 L'analogia con il caso differenziabile	24
5 L'algoritmo	25
6 L'implementazione in Python	33
6.1 Il progetto	33
6.2 Il programma	34
Appendice: il codice	39

Introduzione

La topologia è la branca della matematica che studia le proprietà degli spazi che rimangono invariate quando questi vengono sottoposti a deformazioni continue senza strappi o incollature [2]. Per studiare queste proprietà si usa la topologia algebrica. Essa consiste nel tradurre un complesso problema geometrico in una questione algebrica rigorosa e in molti casi risolvibile, associando agli spazi degli invarianti topologici (come gruppi o numeri). In relazione al tipo di equivalenza considerata, per esempio quella omotopica o quella tramite omeomorfismi, vale che spazi con invarianti distinti non possono in alcun modo essere equivalenti. Tuttavia, il processo inverso non è garantito, poiché due spazi non equivalenti possono comunque condividere gli stessi invarianti. A cavallo tra matematica e informatica si colloca infine la topologia computazionale, branca che si occupa dello studio di strutture dati e algoritmi efficienti per calcolare gli invarianti topologici in modo automatico.

L'ambiente di riferimento standard di questo lavoro è il piano euclideo $\mathbb{R}^2$. I sottospazi topologici di nostro interesse saranno curve poligonali chiuse. Il concetto di continuità per noi si tradurrà nel fatto che non spezzeremo mai le curve poligonali, ma spostando i vertici trasformeremo i lati di conseguenza. Le trasformazioni che andremo a considerare saranno le analoghe discrete delle omotopie regolari per le curve lisce, trasformazioni in cui la derivata rispetto a una data parametrizzazione in ogni punto resta ben definita e non nulla. Per le curve poligonali, la regolarità sarà definita come la condizione secondo la quale i lati consecutivi di una poligonale non devono sovrapporsi. Per stabilire se due curve poligonali chiuse possano essere trasformate l'una nell'altra tramite una simile deformazione, assoceremo a tali curve un invariante numerico chiamato numero di avvolgimento. Esso è infatti invariante per trasformazioni regolari. In questo caso vale anche il viceversa, ovvero se due curve poligonali chiuse hanno lo stesso numero di avvolgimento allora sono regolarmente equivalenti, quindi esso è un invariante completo. Il contenuto della tesi si svilupperà intorno

all'articolo *Constructive Whitney-Graustein Theorem: Or How to Untangle Closed Planar Curves* di Mehlhorn Kurt e Yap Chee-Keng, che ci fornisce un algoritmo per classificare le curve poligonali chiuse tramite un rappresentante canonico, che sarà una n -stella (vedi definizione [3.5](#)). L'obiettivo finale è lo sviluppo di un programma in Python che implementa l'algoritmo. Il programma prende in input una curva poligonale chiusa assegnata come lista di vertici e la trasforma in una n -stella tramite una sequenza di trasformazioni regolari in maniera visiva su un'interfaccia grafica. Questo strumento permette di tradurre la teoria astratta delle omotopie in un'esecuzione visiva tramite la visualizzazione dei vari passaggi intermedi, interattiva e computazionalmente rigorosa.

Capitolo 1

Breve storia del teorema di Whitney-Graustein

1.1 Il teorema

Nel 1937 il matematico statunitense Hassler Whitney pubblicò su *Compositio Mathematica* l'articolo *On regular closed curves in the plane* [4], in cui enunciò il teorema noto poi come teorema di Whitney-Graustein. Per capire meglio l'enunciato, diamo le seguenti definizioni. Considereremo d'ora in poi $\mathbb{R}^2$ dotato del prodotto scalare standard, ovvero il piano euclideo $\mathbb{E}_{\mathbb{R}}^2$. Siano $I = [0, 1]$ l'intervallo chiuso e S^1 la circonferenza unitaria nel piano. Nel seguito differenziabile significherà C^∞ .

Definizione 1.1. Sia $f : I \rightarrow \mathbb{R}^2$ una curva parametrica differenziabile. Diciamo che f è *chiusa* se $f(1) = f(0)$.

Definizione 1.2. Sia $f : I \rightarrow \mathbb{R}^2$ una curva parametrica chiusa differenziabile. Diciamo che f è *regolare* se $f^{(n)}(1) = f^{(n)}(0) \quad \forall n \in \mathbb{N}$ e $f'(t) \neq 0$ per ogni $t \in I$.

Definizione 1.3. Siano $f_0, f_1 : I \rightarrow \mathbb{R}^2$ due curve chiuse. Un'*omotopia differenziabile* tra f_0 e f_1 è una mappa differenziabile $H : I \times I \rightarrow \mathbb{R}^2$ tale che:

- $H(t, 0) = f_0(t)$ per ogni $t \in I$;
- $H(t, 1) = f_1(t)$ per ogni $t \in I$;
- $H(0, s) = H(1, s)$ per ogni $s \in [0, 1]$.

Definizione 1.4. Un'omotopia differenziabile H si definisce *regolare* se, per ogni valore del parametro di deformazione $s \in I$, la curva corrispondente $f_s(t) = H(t, s)$ è una curva regolare.

Definizione 1.5. Due curve parametrizzate chiuse regolari f_0 e f_1 si dicono *regolarmente omotope* se tra esse esiste un'omotopia differenziabile regolare H .

Identificando gli estremi dell'intervallo si ottiene lo spazio quoziente $I/\{0 \sim 1\}$, che risulta naturalmente omeomorfo alla circonferenza unitaria S^1 . Pertanto, d'ora in avanti identificheremo il dominio delle curve chiuse direttamente con S^1 , definendole formalmente come mappe differenziabili $f : S^1 \rightarrow \mathbb{R}^2$. La condizione di regolarità impone che il vettore tangente alla curva non sia mai nullo, ovvero $f'(\theta) \neq 0$ per ogni $\theta \in S^1$. Allora si può definire la seguente:

Definizione 1.6. Si definisce *mappa di Gauss* associata alla curva f l'applicazione continua $T_f : S^1 \rightarrow S^1$ che associa a ogni punto della curva il versore tangente unitario:

$$T_f(\theta) = \frac{f'(\theta)}{\|f'(\theta)\|}$$

È noto dalla topologia algebrica che $\pi_1(S^1) \cong \mathbb{Z}$. Rigorosamente, la definizione del gruppo fondamentale richiederebbe la scelta di un punto base. Tuttavia, poiché il suo gruppo fondamentale è isomorfo a $\mathbb{Z}$, che è un gruppo abeliano, possiamo omettere il punto base dalla notazione e scrivere semplicemente $\pi_1(S^1)$. L'applicazione continua T_f induce un omomorfismo tra i gruppi fondamentali:

$$T_{f*} : \pi_1(S^1) \rightarrow \pi_1(S^1)$$

Poiché $\mathbb{Z}$ è un gruppo ciclico, ogni suo endomorfismo è univocamente determinato da dove viene mandato il suo generatore. Fissata la classe fondamentale $[\omega] \in \pi_1(S^1)$ come generatore (corrispondente all'intero 1), la sua immagine $T_{f*}([\omega])$ dovrà necessariamente essere un multiplo intero del generatore stesso. Esiste perciò un unico intero $W(f)$ tale per cui l'azione dell'omomorfismo assuma la forma:

$$T_{f*}([\omega]) = W(f) \cdot [\omega]$$

Definizione 1.7. L'intero $W(f) \in \mathbb{Z}$ si dice *numero di avvolgimento* della curva liscia f .

Da un punto di vista topologico, esso rappresenta il grado della mappa T_f .

Intuitivamente il numero di avvolgimento calcola il numero algebrico di giri completi che il versore tangente compie in senso antiorario al variare del parametro lungo l'intera curva.

Il risultato principale di Whitney-Graustein è il seguente:

Teorema 1.8 (Teorema di Whitney-Graustein). *Due curve chiuse regolari piane sono regolarmente omotope se e solo se hanno lo stesso numero di avvolgimento.*

In altre parole, il numero di avvolgimento è un invariante completo per curve lisce regolari. Noi non ci occuperemo di curve lisce ma di curve poligonali. La regolarità verrà tradotta con la condizione che lati consecutivi non devono sovrapporsi. Le trasformazioni analoghe alle omotopie regolari saranno invece le trasformazioni regolari (vedi definizione 2.17). Nelle prossime sezioni vedremo il collegamento fra curve lisce regolari con curve poligonali. Verificheremo poi l'equivalenza delle classificazioni quando enunceremo il teorema di Whitney-Graustein nel caso discreto.

1.2 Poligonazione

Il passaggio da una curva liscia regolare ad una poligonale avviene tramite un processo di approssimazione chiamato *partizionamento*. Il processo di partizionamento può avvenire principalmente con due approcci geometrici: l'inscrizione e la circoscrizione.

Nel primo caso, si fissa una partizione ordinata di punti $\{t_0, t_1, \dots, t_n\}$ nel dominio della curva. La poligonale inscritta si costruisce congiungendo i punti campionati consecutivi $f(t_i)$ e $f(t_{i+1})$ tramite segmenti di retta, ovvero le corde della curva. All'infittirsi della partizione, questa poligonale approssima la curva originale.

La poligonale circoscritta viene costruita nel modo seguente: fissata la partizione, i vertici della poligonale sono i punti di intersezione tra le rette tangenti alla curva nei punti $\{f(t_0), f(t_1), \dots, f(t_n)\}$.

La differenziabilità assicura che se la partizione è sufficientemente fine, la curva poligonale approssima molto bene la curva liscia.

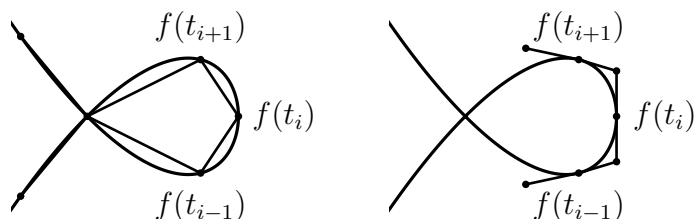


Figura 1.1: Poligonazione tramite inscrizione e circoscrizione.

1.3 Allisciamento

Il processo inverso alla poligonazione è l'allisciamento, un'operazione fondamentale per ricondurre lo studio di una curva poligonale a quello di una curva differenziabile. Con la teoria dell'approssimazione di mappe continue mediante mappe lisce, è sempre possibile approssimare arbitrariamente bene una curva poligonale con una curva di classe C^∞ . Poiché i vertici di una poligonale costituiscono punti in cui la derivata non è definita, il processo di allisciamento interviene localmente in un intorno di ciascun vertice, sostituendo la singolarità angolosa con un arco di curva regolare. In questo caso la finezza dell'approssimazione dipende dall'arco di circonferenza.

Si può vedere che applicando il processo di poligonazione ad una curva liscia e poi quello di allisciamento con finezza sufficiente si ottiene una curva regolarmente omotopa a quella iniziale. Viceversa, applicando il processo di allisciamento ad una curva poligonale e poi quello di poligonazione con finezza sufficiente si ottiene una curva poligonale regolarmente equivalente a quella iniziale.

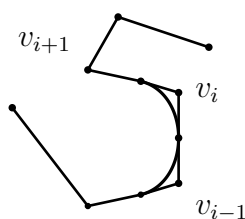


Figura 1.2: Allisciamento.

Capitolo 2

Preliminari

2.1 Poligonal

Definizione 2.1. Una *curva poligonale piana* Π è definita dalla sequenza $\Pi = \langle v_1, v_2, \dots, v_n \rangle$ per $n \geq 2$, dove $v_1, \dots, v_n$ sono punti di $\mathbb{R}^2$ che vengono chiamati *vertici*. Il vertice v_1 si dice *vertice iniziale*, mentre v_n si dice *vertice finale*. Se $n \geq 3$, $v_2, \dots, v_{n-1}$ vengono chiamati *vertici interni*.

Senza perdita di generalità assumeremo che $v_i \neq v_{i+1}$ per $i = 1, \dots, n-1$. Ci riferiremo inoltre alla curva poligonale piana con il termine *poligonale*.

Definizione 2.2. I segmenti congiungenti i vertici $[v_i, v_{i+1}]$ per $i = 1, \dots, n-1$ sono i *lati* della poligonale.

Definizione 2.3. L'inverso della poligonale $\Pi = \langle v_1, \dots, v_n \rangle$ è $\bar{\Pi} = \langle v_n, \dots, v_1 \rangle$.

Definizione 2.4. Una poligonale $\Pi = \langle v_1, \dots, v_n, v_{n+1} \rangle$ si dice *chiusa* se $v_1 = v_{n+1}$, si dice *aperta* altrimenti.

Notazione. Denoteremo una poligonale chiusa $\Pi = \langle v_1, \dots, v_n, v_{n+1} \rangle$ con $P = (v_1, \dots, v_n)$.

D'ora in poi per poligonal

Intuitivamente, vogliamo identificare le poligonal

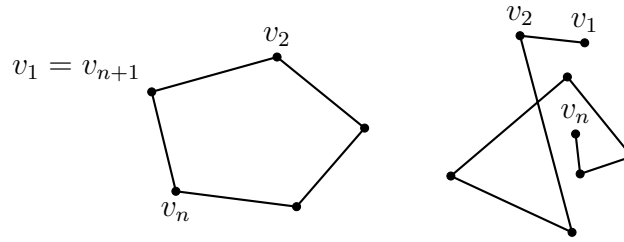


Figura 2.1: Esempi di poligonale chiusa e aperta.

Definizione 2.5. Diciamo che due poligonali chiuse P, P' sono *ciclicamente equivalenti* se $P = (v_1, \dots, v_n)$ e $P' = (v_i, v_{i+1}, \dots, v_n, v_1, \dots, v_{i-1})$ per qualche $i = 1, \dots, n$.

Si verifica facilmente che l'equivalenza ciclica è una relazione d'equivalenza sull'insieme delle poligonali.

Osservazione 2.6. La poligonale, come definita, possiede un'orientazione.

Si può poi definire un'altra tipologia di equivalenza, ovvero una non orientata.

Definizione 2.7. Una *curva poligonale chiusa non orientata* P è la classe di equivalenza di una poligonale chiusa a meno di equivalenza ciclica e inversione.

Denoteremo con $\bar{P}$ il suo inverso.

2.2 Regolarità di poligonali e numero di avvolgimento

Definizione 2.8. Sia v_i un vertice di una poligonale $\langle v_1, \dots, v_n \rangle$. Diciamo che v_i è un *vertice singolare* se i due lati $[v_{i-1}, v_i]$ e $[v_i, v_{i+1}]$ hanno almeno due punti in comune.

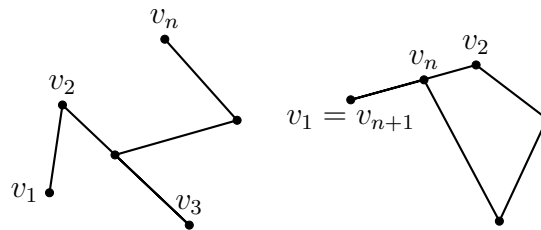


Figura 2.2: Esempi di vertici singolari per poligonali aperte e chiuse. A sinistra v_3 è singolare mentre a destra lo è v_n .

Definizione 2.9. Una poligonale si dice *regolare* se non ha vertici singolari. Si dice *singolare* altrimenti.

Si noti che le condizioni di regolarità non escludono che vertici non consecutivi coincidano o che segmenti non consecutivi si sovrappongano.

Definizione 2.10. Sia v_i un vertice di una poligonale chiusa $P = (v_1, \dots, v_n)$. L'angolo di rotazione denotato con $rot(v_i)$ è definito come l'angolo tale che $|rot(v_i)| \leq \pi$ e $rot(v_i) \equiv \theta_i - \theta_{i-1} \pmod{2\pi}$ dove θ_i è l'angolo appartenente a $[-\pi, \pi]$ che forma il segmento $[v_i, v_{i+1}]$ rispetto a una fissata retta orientata. Considereremo il segno di θ_i positivo se la rotazione è in senso antiorario, negativo altrimenti.

Si noti che dalla definizione segue che una poligonale è regolare se e solo se ogni angolo di rotazione è diverso da $\pm\pi$.

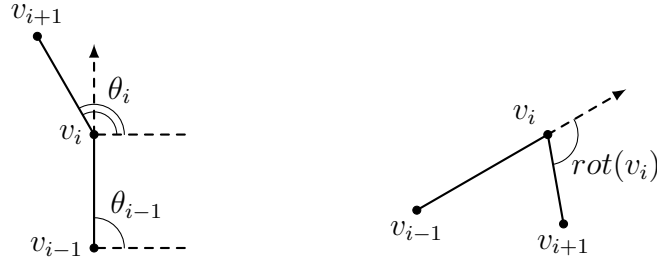


Figura 2.3: A sinistra gli angoli θ_i rispettivi ai vertici, a destra un esempio di angolo di rotazione.

L'angolo di rotazione si può calcolare in valore assoluto tramite la seguente formula:

$$|rot(v_i)| = \arccos \frac{\langle v_i - v_{i-1}, v_{i+1} - v_i \rangle}{\|v_i - v_{i-1}\| \|v_{i+1} - v_i\|} \in [0, \pi]$$

Per determinarne il segno invece bisognerà calcolare il seguente determinante:

$$sgn(rot(v_i)) = sgn(\det(v_i - v_{i-1}, v_{i+1} - v_i))$$

Nel caso i lati si sovrappongano il determinante citato è nullo, tuttavia nella trattazione di poligoni regolari quel caso è influente.

Definizione 2.11. Diremo che una poligonale compie una svolta a sinistra, a destra o non svolta in v_i vertice regolare se rispettivamente il segno dell'angolo di rotazione di quel vertice è positivo, negativo o nullo.

Definizione 2.12. Data una poligonale chiusa regolare P , il suo numero di avvolgimento è definito come segue:

$$W(P) = \frac{1}{2\pi} \sum_{i=1}^n rot(v_i)$$

Proposizione 2.13. *Il numero di avvolgimento di una poligonale chiusa regolare è un numero intero.*

Dimostrazione. Sia $W(P)$ il numero di avvolgimento della poligonale chiusa regolare $P = \langle v_1, \dots, v_n, v_1 \rangle$. Allora tramite la definizione:

$$2\pi W(P) = \sum_{i=1}^n \text{rot}(v_i) \equiv \sum_{i=1}^n \theta_i - \theta_{i-1} \pmod{2\pi}$$

La serie è telescopica, quindi $\sum_{i=1}^n \theta_i - \theta_{i-1} = \theta_n - \theta_0$. Poiché per una poligonale chiusa $\theta_n = \theta_0$, segue subito che $W(P) \in \mathbb{Z}$.

□

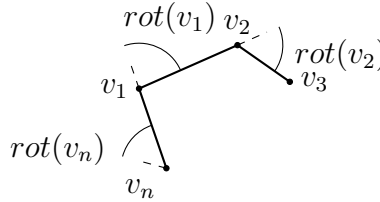


Figura 2.4: Esempio di angoli di rotazione di una poligonale chiusa.

Proposizione 2.14. *Invertendo l'orientazione della poligonale chiusa il numero di avvolgimento cambia segno.*

Dimostrazione. Per definizione la poligonale chiusa inversa $\bar{P}$ è data da $\langle v_1, v_n, \dots, v_2, v_1 \rangle$. Ogni lato $[v_i, v_{i+1}]$ viene percorso nel verso opposto quindi il suo angolo di rotazione rimane invariato in valore assoluto e cambia di segno. Per definizione di numero di avvolgimento se tutti gli angoli di rotazione cambiano di segno, allora $W(\bar{P}) = -W(P)$.

□

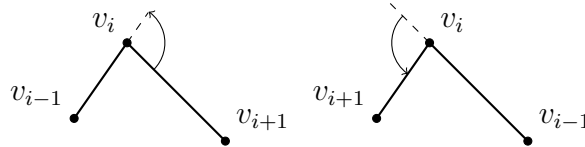


Figura 2.5: Angoli di rotazione dello stesso vertice di poligonali inverse.

Proposizione 2.15. *Per una poligonale chiusa regolare P di n vertici vale la seguente disuguaglianza:*

$$W(P) < \frac{n}{2}$$

Dimostrazione. Tramite la definizione di numero di avvolgimento, e poiché per la regolarità $|rot(v_i)| < \pi$, si ha:

$$|W(P)| = \frac{1}{2\pi} \left| \sum_{i=1}^n rot(v_i) \right| \leq \frac{1}{2\pi} \sum_{i=1}^n |rot(v_i)| < \frac{n}{2}$$

□

Definizione 2.16. Data una poligonale chiusa non orientata P , il suo *numero di avvolgimento* $W(P)$ è definito come il valore assoluto del numero di avvolgimento di qualsiasi poligonale chiusa che lo rappresenta.

2.3 Trasformazioni regolari

Introdurremo ora tre tipologie di trasformazioni elementari di poligonali chiuse regolari (orientate o non orientate), che sono quelle che tratteremo in questa tesi.

Definizione 2.17. Sia $P = (v_1, \dots, v_n)$ una poligonale chiusa regolare e $i = 1, \dots, n$. Definiamo trasformazioni elementari di P le seguenti:

(T_0) **Inserimento.** P può essere trasformata in $Q = (v_1, \dots, v_i, u, v_{i+1}, \dots, v_n)$ dove u è un punto interno a $[v_i, v_{i+1}]$.

(T_1) **Eliminazione.** P può essere trasformata in $Q = (v_1, \dots, v_{i-1}, v_{i+1}, \dots, v_n)$ se v_i è interno al segmento $[v_{i-1}, v_{i+1}]$

(T_2) **Traslazione.** P può essere trasformata in $Q = (v_1, \dots, v_{i-1}, u, v_{i+1}, \dots, v_n)$ dove u è qualsiasi punto tale che per ogni $0 \leq t \leq 1$ la poligonale chiusa $Q_t = (v_1, \dots, v_{i-1}, (1-t)v_i + tu, v_{i+1}, \dots, v_n)$ è regolare. In particolare $Q = Q_1$ è regolare.

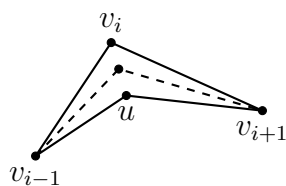


Figura 2.6: Esempio di trasformazione T_2 .

Chiaramente inserimento ed eliminazione sono operazioni inverse. Inoltre il numero di avvolgimento è invariante per T_0 e T_1 poiché il vertice che si aggiunge o si elimina ha angolo di rotazione nullo.

Definizione 2.18. Una *trasformazione regolare* di una poligonale chiusa regolare è una composizione di un numero finito di trasformazioni elementari.

Definizione 2.19. Diciamo che due poligonali chiuse P, Q sono *regolarmente equivalenti* se una può essere trasformata nell'altra tramite un numero finito di trasformazioni regolari.

Si può dimostrare che questa è una relazione di equivalenza.

Teorema 2.20. *Il numero di avvolgimento per poligonali chiuse regolari è un invariante per trasformazioni regolari.*

Dimostrazione. Abbiamo già dimostrato che il numero di avvolgimento è un numero intero e che è invariante per T_0 e T_1 . Poiché per trasformazioni T_2 esso dipende in maniera continua dal parametro t , segue che esso deve essere costante durante la trasformazione. $\square$

Risulta semplice identificare le possibili scelte per il vertice u di T_2 .

Definizione 2.21. Relativamente al vertice v_i , si definiscono i due *coni interdetti* in v_{i-1} e v_{i+1} come segue. Il cono interdetto in v_{i-1} è la regione di piano non contenente v_i e delimitata dalle due semirette aventi origine in v_{i-1} di cui la prima passante per v_{i-2} e la seconda appartenente alla retta passante per v_{i-1} e v_{i+1} non contenente quest'ultimo. Il cono in v_{i+1} è definito in modo analogo.

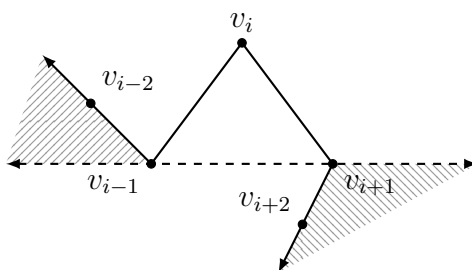


Figura 2.7: Coni interdetti relativi al vertice v_i .

Si vede che la regolarità della poligonale chiusa viene meno se e solo se si effettua una traslazione di v_i nei due coni interdetti.

Capitolo 3

Poligoni stellati

Definizione 3.1. Una poligonale chiusa si dice riducibile se è regolarmente equivalente a una con meno vertici. Se non lo è si dice irriducibile.

Per esempio, si vede facilmente che il triangolo, il poligono a farfalla e la 5-stella sono irriducibili. Inoltre essi hanno numeri distinti di vertici perciò non sono equivalenti fra di loro.

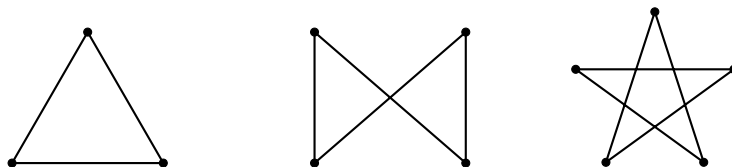


Figura 3.1: Il triangolo, il poligono a farfalla e la 5-stella.

Lemma 3.2. *Ogni poligonale chiusa P può essere trasformata tramite trasformazioni T_2 in una poligonale chiusa Q avente i vertici a due a due distinti e che giacciono su una data circonferenza C contenente P .*

Dimostrazione. Sia C tale circonferenza. Considero un vertice v di P e il cono non interdetto di v , ovvero il complementare dell'unione dei due coni interdetti. Si vede che contiene un cono aperto non vuoto di infinite semirette con origine v . Esse intersecano la circonferenza in un punto u . Sia quindi T_2 tale che porta v in u . Possiamo scegliere la semiretta tale per cui u è distinto dagli altri vertici. Si può ripetere il ragionamento per tutti i vertici. $\square$

Osservazione 3.3. Una poligonale chiusa i cui vertici giacciono su una circonferenza è regolare se e solo se $v_{i-1} \neq v_{i+1} \forall i$. Segue che se i vertici sono a due a due distinti allora la poligonale chiusa è regolare.

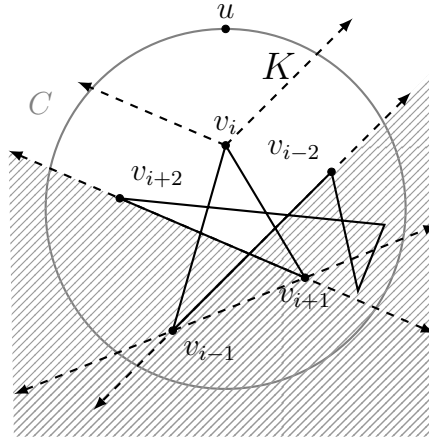


Figura 3.2: Cono K relativo a v_i nel lemma 3.2.

Notazione. Sia $\Pi_1 = \langle v_1, \dots, v_n \rangle$ una poligonale. Denoto con Π_i per $i = 1, \dots, n$ la poligonale $\Pi_i = \langle v_i, v_{i+1}, \dots, v_n, v_1, v_2, \dots, v_{i-1} \rangle$.

Definizione 3.4. Una poligonale $\Pi = \langle v_1, \dots, v_n \rangle$ si dice poligonale stellata se ogni lato $e_i = [v_i, v_{i+1}]$, per $i \in \{1, \dots, n-1\}$, interseca ognuno dei lati precedenti $e_1, e_2, \dots, e_{i-1}$.

Definizione 3.5. Una poligonale chiusa $P = (v_1, \dots, v_n)$ si chiama n -stella se per qualche scelta di i , $i \in \{1, \dots, n\}$ uno dei suoi rappresentanti Π_i è stellato.

Il triangolo e il poligono a farfalla sono rispettivamente una 3-stella e una 4-stella.

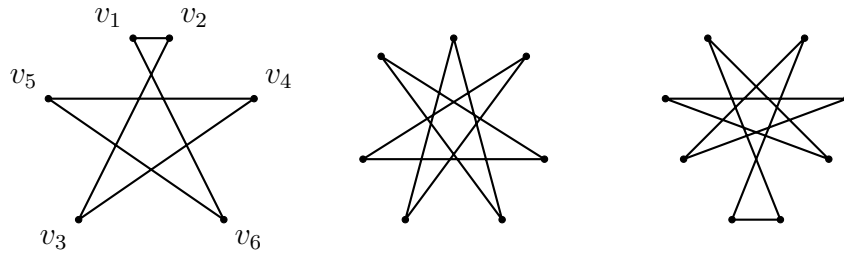


Figura 3.3: Esempi di 6-stella, 7-stella e 8-stella.

Nella figura si vede che, per esempio, il rappresentante stellato della prima figura è Π_1 .

Esempio 3.6 (Esempio di riduzione.). Riduciamo la 6-stella. Si vede che si può traslare tramite trasformazioni regolari v_3 nel punto medio del segmento di estremi v_4 e v_2 . Tramite una trasformazione T_0 possiamo ora eliminarlo. Facciamo analogamente per v_2 . Abbiamo ottenuto così un poligono a farfalla, che è irriducibile.

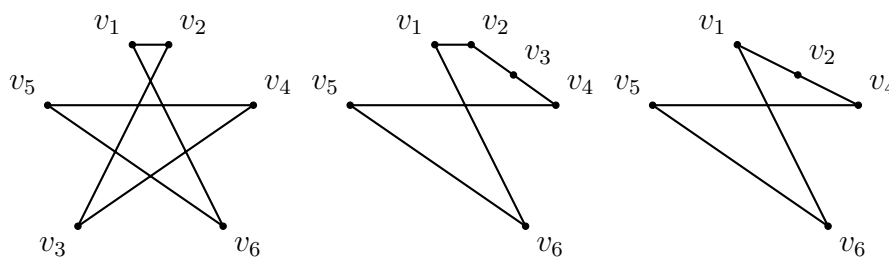


Figura 3.4: Riduzione della 6-stella.

Definizione 3.7. Una poligonale si dice sinistrorsa (rispettivamente destrorsa) se compie solo svolte a sinistra, ossia $rot(v_i) > 0 \forall i$ (rispettivamente a destra, ossia $rot(v_i) < 0 \forall i$).

Proposizione 3.8. Una poligonale stellata per $n \geq 3$ è o destrorsa o sinistrorsa.

Dimostrazione. Assumiamo che i primi tre vertici della poligonale non siano allineati. Allora l'angolo di rotazione di v_2 è positivo o negativo. Consideriamo il caso in cui è positivo. Procedo per induzione sull'indice i dei vertici. Suppongo che la poligonale stellata sia destrorsa fino all'indice i , devo dimostrare allora che lo è anche per $i + 1$. Se per assurdo non lo fosse, vorrebbe dire che i compie una svolta a sinistra. Ma allora il lato $[v_i, v_{i+1}]$ appartenerrebbe al semipiano formato dalla retta passante per v_{i-1} e v_i opposto rispetto al lato $[v_{i-2}, v_{i-1}]$. Allora non potrebbe intersecarlo, assurdo. Si procede analogamente nel caso dell'angolo di rotazione negativo. $\square$

Osservazione 3.9. Sia $\Pi = \langle v_1, \dots, v_n \rangle$ una poligonale stellata. Se n è dispari, Π_i è ancora una poligonale stellata per ogni indice i . Se n è pari e $n \neq 4$, Π_i è una poligonale stellata solo per $i = 1$. Per $n = 4$, Π_1 e Π_3 sono le uniche poligonali stellate. La giustificazione formale seguirà da lemmi successivi.

Definizione 3.10. Siano $u_1, \dots, u_k$ vertici distinti di una poligonale chiusa P giacenti su una circonferenza C con $k \geq 3$. Chiamiamo *ordine ciclico* dei vertici $u_1, \dots, u_k$ una sequenza della forma

$$u_1 < u_2 < \dots < u_k$$

se percorrendo la circonferenza C in senso orario partendo da u_1 incontriamo in ordine $u_1, u_2, \dots, u_k$. Chiaramente $u_1 < u_2 < \dots < u_k$ è equivalente a $u_2 < u_3 < \dots < u_k < u_1$ ecc.

Lemma 3.11. Sia $\Pi = \langle v_1, \dots, v_n \rangle$ una poligonale stellata con i vertici giacenti su una circonferenza. Allora:

- Π è *sinistrorsa* se e solo se induce i seguenti ordini ciclici sui vertici:

$$v_1 < v_3 < v_5 < \dots < v_{dispari} < v_2 < v_4 < v_6 < \dots < v_{pari}$$

dove $v_{dispari} = v_n$, $v_{pari} = v_{n-1}$ se n è *dispari* e $v_{dispari} = v_{n-1}$, $v_{pari} = v_n$ se n è *pari*.

- Π è *destrorsa* se e solo se induce i seguenti ordini ciclici sui vertici:

$$v_{dispari} < \dots < v_5 < v_3 < v_1 < v_{pari} < \dots < v_6 < v_4 < v_2$$

dove $v_{dispari} = v_n$, $v_{pari} = v_{n-1}$ se n è *dispari* e $v_{dispari} = v_{n-1}$, $v_{pari} = v_n$ se n è *pari*.

Dimostrazione. Vediamo il caso sinistrorso procedendo per induzione su n . Il caso base è per $n = 5$, ovvero una 5-stella inscritta in una circonferenza. Se la poligonale è sinistrorsa allora l'ordine ciclico è sicuramente $v_1 < v_3 < v_5 < v_2 < v_4$. Supponiamo che per $n - 1$ valga la relazione d'ordine ciclico del lemma. Sia n pari. Ci basta dimostrare che $v_{n-2} < v_n < v_1$. Poiché la poligonale è sinistrorsa considerando la svolta in v_{n-1} deduciamo $v_{n-2} < v_n < v_{n-1}$. Inoltre se fosse $v_1 < v_n < v_{n-1}$ il lato $[v_{n-1}, v_n]$ non intersecherebbe il lato $[v_1, v_2]$, assurdo. Gli altri casi sono analoghi. $\square$

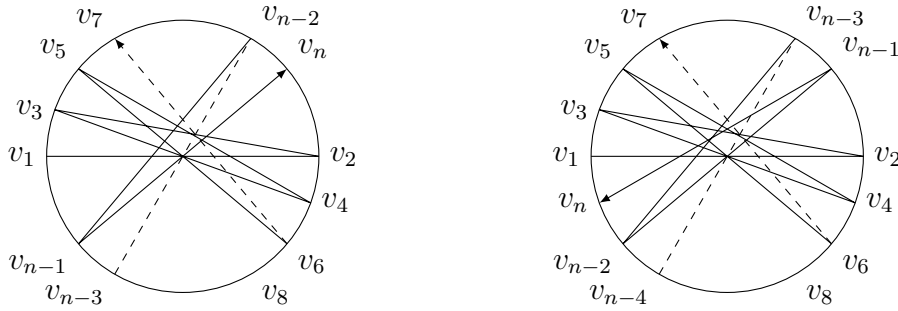


Figura 3.5: A sinistra il caso v_n dispari, a destra v_n pari.

3.1 Lemmi tecnici

Sia $P = (u_1, \dots, u_n)$ una poligonale chiusa i cui vertici giacciono su una circonferenza C . Sia x un qualsiasi punto di C diverso da u_1 . Sia $A \subseteq C$ uno dei due archi di circonferenza di estremi u_1 e x .

Definizione 3.12. Diciamo che un lato della poligonale chiusa è *attivo* se almeno uno dei suoi estremi appartiene ad A .

Definizione 3.13. Due vertici di P che appartengono ad A si dicono *collegati* se esiste una sottopoligonale di lati attivi che li congiunge.

Osservazione 3.14. Essere collegati è una relazione d'equivalenza. I vertici di P che appartengono ad A sono quindi partizionati in classi di equivalenza che chiamiamo *attive*.

Lemma 3.15. Sia U un'unione di classi attive tale che $u_1 \in U$ e se x è vertice di P supponiamo $x \in U$. Allora per ogni intorno aperto N di x in C esiste una sequenza di $|U|$ traslazioni che manda u_1 in x , manda i vertici in $U \setminus \{u_1\}$ in $N \setminus A$, e mantiene fissi i rimanenti vertici di P . Inoltre l'ordine ciclico indotto da C sugli elementi di U si preserva rispetto tali traslazioni.

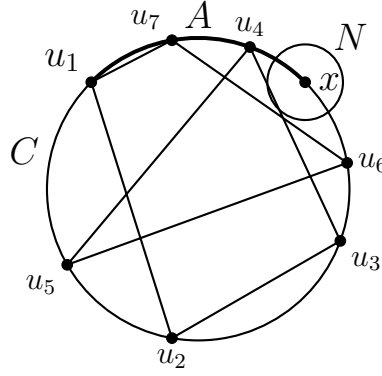


Figura 3.6: In questo disegno la componente connessa di u_1 contiene u_2, u_6 e u_7 .

Dimostrazione. Dalla definizione si ha che $U \subseteq A$. I vertici di U possono essere ordinati in base a distanza crescente da x misurata lungo l'arco A . Cominciando dal vertice in U più vicino a x , trasliamo ognuno in $N \setminus A$. L'ultimo vertice da traslare è u_1 , che può essere traslato in x . Questo si può fare perché u_j può essere traslato in un qualsiasi punto di C tale che l'indotta permutazione ciclica su $u_{j-2}, u_{j-1}, u_j, u_{j+1}, u_{j+2}$ viene preservata. Quando trasliamo $u_j \in U$ come descritto si vede che ognuno fra $u_{j-2}, u_{j-1}, u_j, u_{j+1}, u_{j+2}$ è già stato traslato in $N \setminus A$ o si trova fuori da A . In ogni caso possiamo traslare u_j in qualche punto di $N \setminus A$ preservando l'indotta permutazione di U . $\square$

Notazione. Per riferirci a questo tipo di trasformazione diremo che trasliamo u_1 in x relativamente a (A, U, N) .

Osserviamo che la più piccola scelta per U è l'unione delle classi attive di u_1 e delle classi attive di x se $x \in U$. In questo caso definiamo la composizione delle

traslazioni effettuate come *traslazione debole* di u_1 in x . La più grande scelta per U è invece l'insieme contenente tutti i vertici di P che appartengono ad A . In quest'altro caso ci riferiremo all'insieme delle traslazioni come *traslazione forte* di u_1 in x . Inoltre, diremo che la traslazione è oraria o antioraria a seconda di quale dei due archi scegliamo come A .

D'ora in poi assumeremo quindi che le poligoni chiuse abbiano i vertici giacenti su una circonferenza.

Lemma 3.16. *Siano $P = (u_1, \dots, u_n)$ e $Q = (v_1, \dots, v_n)$ due poligoni chiuse con i vertici giacenti sulla stessa circonferenza C . Supponiamo che C induca lo stesso ordine ciclico sugli indici di P e Q . Allora P e Q sono regolarmente equivalenti in al più $\frac{3}{2}n$ trasformazioni elementari.*

Dimostrazione. Se u_1 e v_1 sono distinti, tramite il lemma applichiamo una traslazione forte da u_1 a v_1 in senso orario o antiorario. Scegliamo quella per cui si debbano effettuare al più $n/2$ traslazioni. Usando di nuovo il lemma applichiamo di nuovo una traslazione forte da u_2 a v_2 , questa volta dobbiamo effettuare una sola traslazione. Iteriamo il procedimento traslando tutti i vertici u_i in v_i . Considerando che $u_1, \dots, u_{i-1}$ sono coincidenti con $v_1, \dots, v_{i-1}$ e queste traslazioni si fanno in un solo passo ciascuno si ha la tesi. $\square$

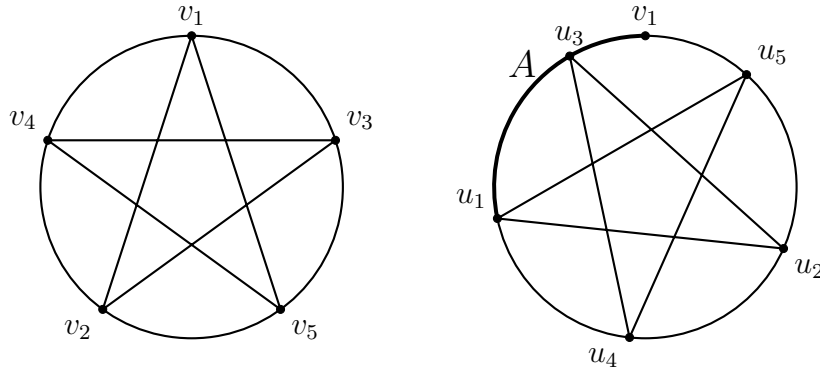


Figura 3.7: Lemma 3.16.

Corollario 3.17. *Due n -stelle con i vertici giacenti sulla stessa circonferenza C sono regolarmente equivalenti.*

Dimostrazione. La circonferenza C induce sulle n -stelle gli stessi ordini ciclici per il lemma 3.11. $\square$

Proposizione 3.18. *Sia P una n -stella.*

- Se $n = 4$, allora $W(P) = 0$.
- Se n è dispari, allora $W(P) = \pm \frac{n-1}{2}$, dove il segno dipende dall'orientazione.

Dimostrazione. Se $n = 4$ si vede facilmente che gli angoli di rotazione si annullano a vicenda. Sia n dispari. Supponiamo la n -stella sia sinistrorsa. Per il lemma precedente tutte le n -stelle sono regolarmente equivalenti quindi hanno lo stesso numero di avvolgimento. Consideriamo la n -stella con i vertici equidistanziati, si ha che gli angoli alla circonferenza valgono $\frac{\pi}{n}$, gli angoli di rotazione valgono quindi $\pi - \frac{\pi}{n}$. Allora il numero di avvolgimento vale

$$W(P) = \frac{1}{2\pi} \sum_{i=1}^n \text{rot}(v_i) = \frac{1}{2\pi} n \left(\pi - \frac{\pi}{n} \right) = \frac{n-1}{2}$$

Il caso destrorso è analogo cambiando i segni degli angoli di rotazione. $\square$

A questo importante risultato va aggiunto l'ultimo lemma di questa sezione.

Lemma 3.19. *Siano n, m numeri interi positivi dispari oppure 4. Se $n \neq m$, allora una n -stella e una m -stella non sono regolarmente equivalenti.*

Dimostrazione. Abbiamo calcolato nella proposizione 3.18 che il numero di avvolgimento della n -stella per $n = 4$ è 0 mentre per ogni intero positivo dispari n è $\frac{n-1}{2}$. La tesi segue dal fatto che il numero di avvolgimento di una poligonale chiusa rimane invariato tramite trasformazioni regolari. $\square$

Il corollario 3.17 ci assicura che si può scegliere un rappresentante univoco per ogni n -stella e il lemma 3.19 ci fornisce una lista infinita di poligoni chiusi non equivalenti. Nel seguito, considereremo come forma canonica di una n -stella la configurazione in cui gli n vertici sono disposti in modo equidistante lungo la circonferenza. La posizione dei vertici coincide quindi esattamente con quella di un poligono regolare convesso a n lati. Denoteremo questa n -stella canonica con Σ_n . L'obiettivo finale sarà dimostrare che ogni classe di equivalenza regolare è rappresentata da una e una sola di queste poligoni chiuse. Nel prossimo capitolo faremo infatti vedere che ogni poligonale chiusa è equivalente ad una n -stella per qualche n dispari o uguale a 4.

Capitolo 4

Il teorema di Whitney-Graustein poligonale

4.1 Il teorema

In questo capitolo considereremo solo trasformazioni T_1 e T_2 ; il ruolo di T_0 resta comunque fondamentale per la definizione di relazione d'equivalenza. Il risultato principale di questo capitolo è quello di Mehlhorn e Yap, ovvero il seguente:

Teorema 4.1. *Ogni poligonale chiusa regolare è regolarmente equivalente ad una n -stella per un certo n che è dispari oppure uguale a 4.*

Corollario 4.2. *Una n -stella è irriducibile se e solo se $n = 4$ o n è dispari.*

Dimostrazione. Se fosse n pari e maggiore di 4, applicando il teorema potremmo trasformarla in una m -stella con m dispari. Se fosse $m < n$ la n -stella sarebbe riducibile ottenendo una contraddizione. Sia allora $m > n$. Per la disuguaglianza della proposizione 2.15 si ha la seguente contraddizione:

$$|W(P)| = |W(\Sigma_m)| = \frac{m-1}{2} < \frac{n}{2}$$

Viceversa se $n = 4$ la n -stella è sicuramente irriducibile. Sia allora Σ_n la n -stella con n dispari riducibile ad una P avente $m < n$ vertici.

Per la proposizione 3.18 il numero di avvolgimento della n -stella è:

$$|W(\Sigma_n)| = \frac{n-1}{2}$$

Poiché il numero di avvolgimento rimane invariato:

$$|W(\Sigma_n)| = |W(P)| = \frac{n-1}{2}$$

Tramite la disuguaglianza della proposizione 2.15 otteniamo:

$$|W(P)| < \frac{m}{2}$$

Unendo le due condizioni, giungiamo alla disuguaglianza:

$$\frac{n-1}{2} < \frac{m}{2}$$

quindi $n < m + 1$, otteniamo un assurdo.

□

Ricordiamo che il teorema di Whitney-Graustein 1.8 ci dice che due curve lisce regolari qualsiasi sono regolarmente omotope se e solo se hanno lo stesso numero di avvolgimento. La formulazione analoga del teorema nel caso poligonale è quindi la seguente:

Teorema 4.3 (Teorema di Whitney-Graustein poligonale.). *Due curve poligonali chiuse regolari sono regolarmente equivalenti se e solo se hanno lo stesso numero di avvolgimento.*

I due teoremi ci dicono che le classificazioni nel caso differenziabile e nel caso poligonale coincidono. Mehlhorn e Yap ci danno inoltre un rappresentante canonico per ogni classe di equivalenza regolare.

Notazione. Per brevità d'ora in poi denoteremo i vertici con gli indici dei vertici. Scriveremo perciò $P = (1, \dots, n)$ per una poligonale chiusa di n vertici. Gli ordini sugli indici saranno quelli indotti dai vertici.

Di seguito diamo un fondamentale criterio per eliminare un vertice.

Lemma 4.4. *Sia $P = (1, \dots, n)$ con $n \geq 5$ tale che i lati $[1, 2]$ e $[3, 4]$ non si intersecano, e i lati $[2, 3]$ e $[4, 5]$ non si intersecano. Allora P è equivalente a $(1, 2, 4, 5, \dots, n)$ tramite una trasformazione T_2 seguita da una T_1 . In altre parole si può eliminare il vertice 3.*

Dimostrazione. Le condizioni di non-intersezione implicano che il triangolo $\triangle 234$ è contenuto nel cono non interdetto del vertice 3. Allora trasliamo 3 nel

punto medio di $[2, 4]$ tramite una trasformazione T_2 . A questo punto possiamo applicare T_1 per eliminare il vertice 3. $\square$

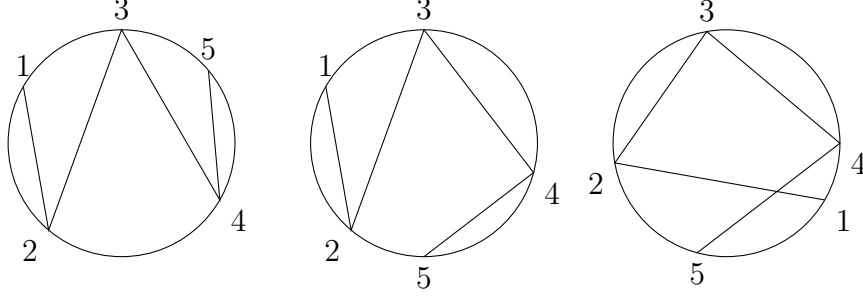


Figura 4.1: Esempi del lemma 4.4.

In preparazione alla dimostrazione del teorema principale di questo capitolo, introduciamo le seguenti definizioni e lemmi preliminari.

Definizione 4.5. Diciamo che una poligonale chiusa $P = (1, \dots, n)$ con i vertici giacenti su una circonferenza contiene una N -forma se $n \geq 4$ e per qualche scelta dell'indice i abbiamo che:

$$i < i + 1 < i + 3 < i + 2$$

oppure

$$i < i + 2 < i + 3 < i + 1$$

Chiameremo $(i, i + 1, i + 2, i + 3)$ N -forma.

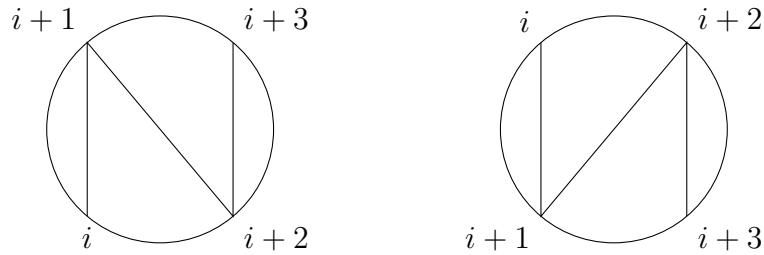


Figura 4.2: N -forme.

Lemma 4.6. Una poligonale chiusa irriducibile $P = (1, \dots, n)$ non contiene N -forme a meno che $n = 4$.

Dimostrazione. Per assurdo sia P irriducibile contenente una N -forma e $n \geq 5$. A meno di rinominare ciclicamente i vertici possiamo assumere $1 < 2 < 4 < 3$. Poiché P è irriducibile e il lato $[1, 2]$ non si interseca con $[3, 4]$, per il lemma

precedente il lato $[4, 5]$ deve intersecare il lato $[2, 3]$. Si ottiene dunque che $3 < 5 < 2$. Con analoghe considerazioni si giunge a $2 < n < 3$. Segue che n deve essere diverso da 5. Sia allora $n \geq 6$. Se $1 < 5 < 2$ possiamo traslare 2 affinché $1 < 2 < 5$ poiché $2 < n < 3$. Ma allora avremmo che 1 e 5 non sono contenuti nel triangolo $\triangle 234$ e quindi potremmo eliminare 3, ottenendo una contraddizione. Perciò si deve avere $3 < 5 < 1$. Per simmetria si ottiene $2 < n < 4$. Se $n = 6$ si vede che la poligonale è riducibile. Altrimenti consideriamo i due casi. Il primo è se $n < 6 < 4$. Se fosse $5 < 7 < 1$ potremmo eliminare 5, ottenendo una contraddizione. Allora possiamo traslare 5 per ottenere $1 < 5 < 2$, che si riduce al caso visto in precedenza. Nell'altro caso $6 < n < 4$ si può traslare 4 in modo che $4 < n < 3$ e quindi $4 < n < 2$. Anche questo si riconduce al caso precedente. $\square$

Si osservi che una poligonale chiusa non contiene N -forme se e solo se la sua versione orientata è sinistrorsa o destrorsa.

Definiamo ora un altro tipo di configurazione particolare che possono assumere i vertici delle poligonali chiuse.

Definizione 4.7. Diciamo che una poligonale chiusa $P = (1, \dots, n)$ contiene una U -forma se $n \geq 4$ e per qualche scelta dell'indice i abbiamo che:

$$i < i + 1 < i + 2 < i + 3$$

oppure

$$i < i + 3 < i + 2 < i + 1$$

Chiameremo $(i, i + 1, i + 2, i + 3)$ una U -forma.

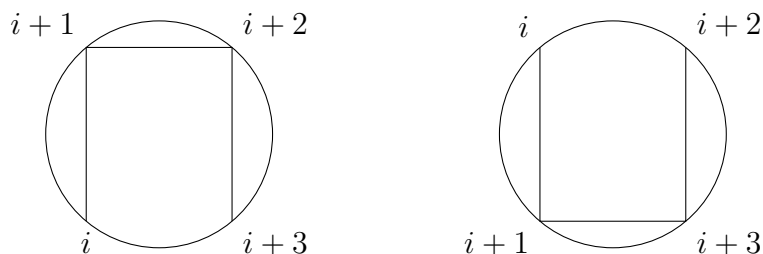


Figura 4.3: U -forme.

Lemma 4.8. Una poligonale chiusa irriducibile $P = (1, \dots, n)$ non contiene U -forme.

Dimostrazione. Si vede facilmente che per $n = 4, 5$ una U -forma implica la riducibilità; sia allora $n \geq 6$. Senza perdita di generalità supponiamo che $(2, 3, 4, 5)$ sia una U -forma e $2 < 5 < 4 < 3$. Poiché 3 non può essere eliminato deve essere $4 < 1 < 3$, analogamente con 4 si ottiene $4 < 6 < 3$. Supponiamo che sia $4 < 1 < 6 < 3$. Potremmo traslare 3 affinché $1 < 3 < 6$ e quindi eliminare 4, una contraddizione. Abbiamo allora $4 < 6 < 1 < 3$. La poligonale è left-turning poiché non contiene N -forme, quindi per ogni indice i si ha $i < i - 1 < i + 1$. In particolare $1 < n < 2$ e $5 < 7 < 6$, ciò implica che $n \geq 8$. Sia A l'arco da 5 a 4 in senso orario che contiene 5 ma non 4. Analogamente sia B l'arco da 6 a 1 in senso orario che contiene 6 ma non 1. Si noti che possiamo traslare 7 in A se non gli appartiene. Se l'indice 8 non appartenesse a B potrei traslare 6 affinché $4 < 1 < 6 < 3$, una contraddizione. Allora 8 appartiene a B e quindi $n \neq 8$ perché deve essere $1 < n < 2$. Se fosse $n = 9$ si avrebbe $7 < 9 < 8$, segue che $n \geq 10$. Supponiamo per induzione che: (1) per qualche $i \geq 7$ dispari gli indici $5 < 7 < \dots < i - 2 < i$ appartengano ad A , (2) gli indici $6 < 8 < \dots < i - 1 < i + 1$ appartengono a B , e (3) $n \geq i + 3$. Vogliamo dimostrare che possiamo estendere le ipotesi. Per prima cosa possiamo traslare $i + 2$ in A . Poi, se $i + 3$ non appartiene a B sia x tale che $1 < x < 3$ e $1 < x < i + 3$. Effettuiamo una traslazione debole di 6 in senso orario in x tramite il lemma. Ma allora $4 < 1 < 6 < 3$, una contraddizione, perciò $i + 3$ appartiene a B . Poiché $i + 3$ sta in B si ha $n \geq i + 4$. Ma $i + 2 < i + 4 < 1$, allora $n \geq i + 5$. Poiché P ha un numero finito di vertici si ha un assurdo. $\square$

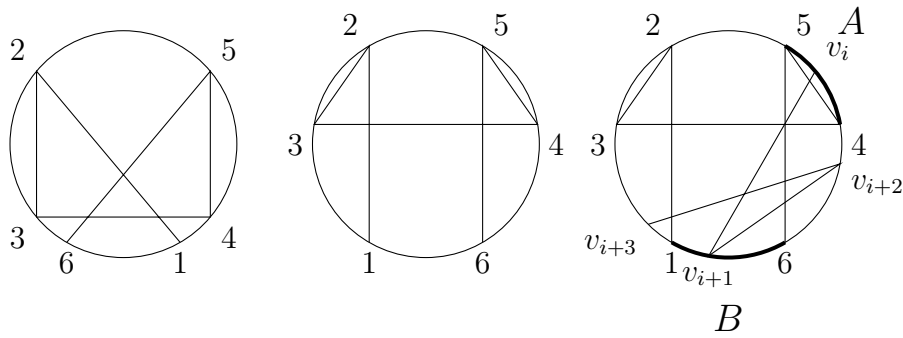


Figura 4.4: Eliminazione di U -forma.

Concludiamo infine con un ultimo lemma.

Lemma 4.9. *Sia $P = (1, 2, \dots, n)$ una poligonale chiusa irriducibile. Allora $\Pi = \langle 1, 2, \dots, n \rangle$ è una poligonale stellata.*

Dimostrazione. L'enunciato è banalmente vero per $n = 3$ e $n = 4$. Dai lemmi sappiamo che P non contiene né N -forme né U -forme. Dimostreremo che se $\Pi_v = \langle 1, 2, \dots, v \rangle$ per $v = 3, \dots, n - 1$ è una poligonale stellata allora Π_{v+1} è una poligonale stellata. Senza perdita di generalità assumiamo $1 < 3 < 2$. La poligonale Π_v è sempre stellata per $v = 3$. Sia quindi $v \geq 5$. Sia v dispari. Se per assurdo Π_{v+1} non fosse una poligonale stellata avremmo $1 < v + 1 < v - 2$ (si vede che altrimenti violeremmo la condizione di irriducibilità). Sia x un punto tale che $1 < x < 3$ e $1 < x < v + 1$. Effettuiamo una traslazione debole in senso orario da 4 a x . Si osservi che $[2, 3]$ e $[v, v + 1]$ non sono lati attivi, quindi l'indice 1 rimane fissato. Segue che $(1, 2, 3, 4)$ è una U -forma, ottenendo una contraddizione. Sia allora v pari. Se Π_{v+1} non è una poligonale stellata, abbiamo $2 < v + 1 < v - 2$. Sia x un punto tale che $2 < x < v + 1$ e A l'arco di circonferenza da $v - 2$ a x in senso orario. Effettuiamo una traslazione debole da $v - 2$ a x . Notiamo che i lati $[1, 2]$ e $[v - 1, v]$ non sono attivi. Allora $v + 1$ rimane fissato. Segue che $(v - 2, v - 1, v, v + 1)$ è una U -forma, ottenendo una contraddizione. $\square$

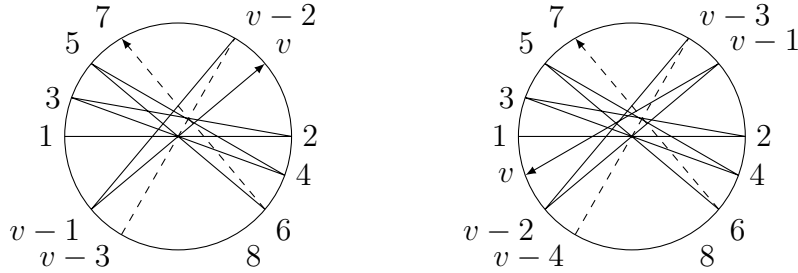


Figura 4.5: A sinistra il caso v dispari, a destra v pari.

Data una poligonale chiusa P possiamo quindi effettuare trasformazioni T_1 e T_2 finché diventa irriducibile. Infine, per il lemma appena dimostrato, deve essere una n -stella.

4.2 L'analogia con il caso differenziabile

La vicinanza metrica assicura la stabilità topologica: il processo di smussamento di una curva poligonale chiusa preserva la classe di omotopia regolare. Di conseguenza, la curva differenziabile ottenuta eredita esattamente lo stesso numero di avvolgimento della poligonale originaria, stabilendo un ponte formale rigoroso tra la versione analitica del Teorema di Whitney-Graustein e la versione poligonale.

Capitolo 5

L'algoritmo

La dimostrazione del teorema di Whitney-Graustein poligonale vista nel capitolo precedente è di tipo costruttivo. Mehlhorn e Yap producono un algoritmo implicito di complessità computazionale quadratica basato su quest'ultima. In questo capitolo descriveremo quindi il processo da eseguire per portare una poligonale chiusa regolare con i vertici giacenti su una circonferenza in forma canonica, ovvero la trasformeremo in una n -stella.

L'algoritmo prevede di procedere in ordine sui vertici. Durante il procedimento avremo quindi già modificato i vertici $v_1, \dots, v_{j-1}$, e la poligonale regolarmente equivalente sarà fatta così:

$$P_j = (u_1, u_2, \dots, u_{i-1}, v_j, \dots, v_n)$$

con $\langle u_1, u_2, \dots, u_{i-1} \rangle$ poligonale stellata sinistrorsa. In questo algoritmo trasformeremo infatti tutte le poligonali in poligonali stellate sinistrorse. Si osservi che $i \leq j$, poiché nel caso la poligonale venga ridotta ad una con meno vertici vale la disuguaglianza stretta.

Per $i = 1, 2, 3$ porremo direttamente $u_i = v_i$, l'algoritmo processerà quindi i vertici v_j partendo da $j = 4$.

Spieghiamo ora il procedimento da applicare al vertice v_j , assumendo quindi che $\langle u_1, \dots, u_{i-1} \rangle$ sia una poligonale stellata.

Algoritmo sul vertice v_j

Vediamo il caso in cui i è dispari.

Caso A. In $i - 1$ la poligonale $\langle u_1, \dots, u_{i-2}, u_{i-1}, v_j, v_{j+1}, \dots, v_n \rangle$ compie una

svolta a destra.

Caso A1. In j la poligonale compie una svolta a sinistra. In questo caso possiamo eliminare il vertice $i - 1$.

Giustificazione. Abbiamo dimostrato nel lemma 4.4 che se i vertici assumono questa configurazione, il vertice $i - 1$ è eliminabile.

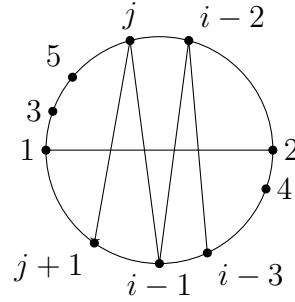


Figura 5.1: Caso A1.

Caso A2. In j la poligonale compie una svolta a destra. Questa volta dobbiamo considerare tre sottocasi.

Caso A21. $j < j + 1 < 2$. Possiamo eliminare l'indice $i - 1$ dopo al massimo una traslazione.

Giustificazione. Se $j < j + 1 < i - 2$ possiamo eliminare $i - 1$. Altrimenti possiamo effettuare una traslazione debole di $i - 2$ in un punto tale che $j + 1 < i - 2 < 2$ e quindi eliminare $i - 1$.

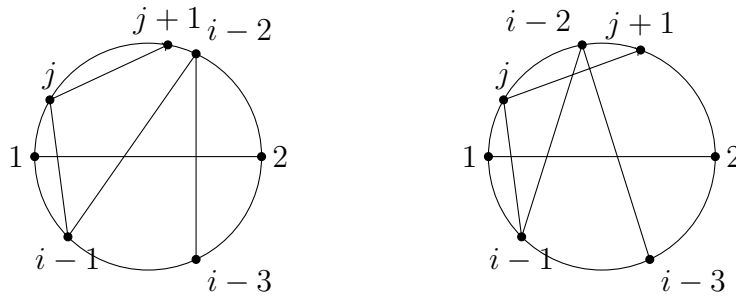


Figura 5.2: Casi A21.

Caso A22. $j < 2 < j + 1 < 1$. Con $O(j)$ traslazioni possiamo eliminare l'indice $i - 2$.

Giustificazione. L'indice $i - 4$ deve essere tale che $i - 4 < i - 2 < i - 3$, infatti assumiamo che la poligonale fino a j sia sinistrorsa. Sposto l'indice $i - 2$ con una traslazione debole in un intorno di 2 e l'indice $i - 4$ in un intorno di 1. Allora $i - 4 < j < i - 2$. Queste mosse possono comportare $O(j)$ traslazioni. Possiamo poi traslare l'indice $i - 1$ in senso orario affinché $i - 4 < i - 1 < j$ (si può fare perché in j c'è una svolta a destra). Possiamo allora eliminare l'indice $i - 2$.

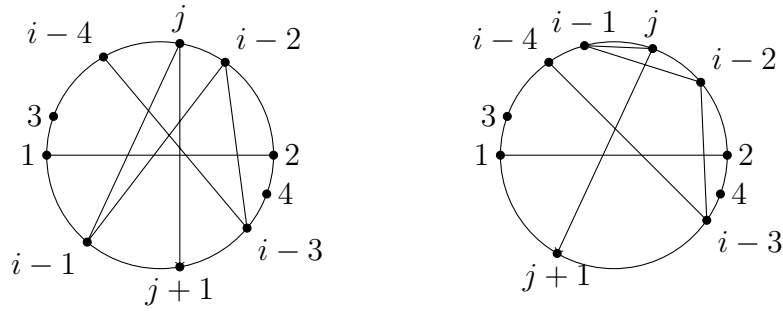


Figura 5.3: Caso A22.

Caso A23. $j < 1 < 2 < j + 1$. Consideriamo due sottocasi.

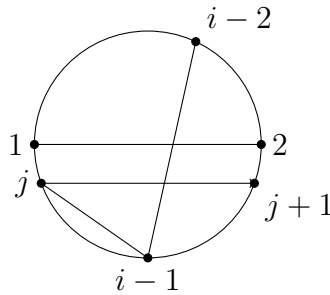


Figura 5.4: Caso A23.

Caso A231. $j < 1 \leq j + 2$.

Giustificazione. Possiamo traslare j in senso orario affinché $1 < j < 2$ e ci riportiamo al caso A22.

Caso A232. $j < j + 2 < 1$. Consideriamo i due sottocasi dipendenti da $j + 3$.

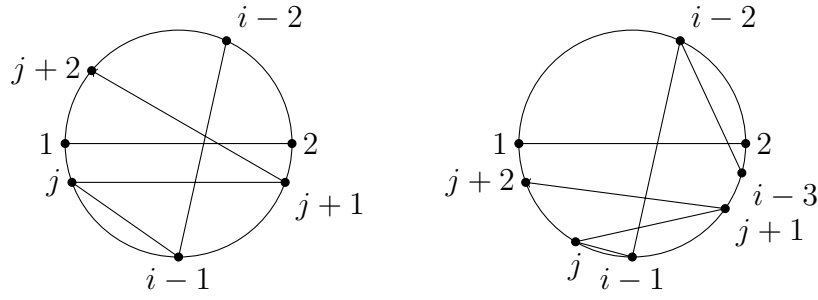


Figura 5.5: Casi A231, A232.

Caso A2321. La poligonale in $j+2$ compie una svolta a destra. Si possono eliminare in ordine gli indici $j+1$ e j .

Giustificazione. Possiamo traslare i vertici citati nei punti medi dei lati ed eliminarli.

Caso A2322. La poligonale in $j+2$ compie una svolta a sinistra. Dopo qualche traslazione possiamo rimpiazzare l'indice $i-1$ con $j+2$ ed incrementare j di 3.

Giustificazione. Trasliamo $j+1$ in senso antiorario in modo che $1 < j+1 < i-2$. Possiamo eliminare $i-1$ e j , e poi anche $j+1$.

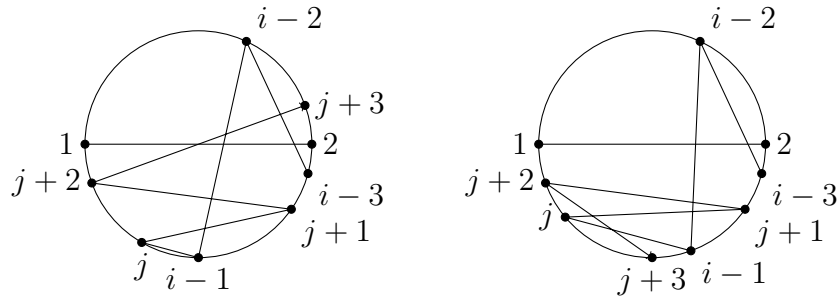


Figura 5.6: Casi A2321, A2322.

Caso B. In $i-1$ la poligonale $\langle u_1, \dots, u_{i-2}, u_{i-1}, v_j, v_{j+1}, \dots, v_n \rangle$ compie una svolta a sinistra. Abbiamo due sottocasi a seconda che j appartenga all'arco in senso orario di estremi $i-2$ e 2 .

Caso B1. $j < 2 < i-2$. La poligonale fino a j è stellata, incrementiamo i e j di 1.

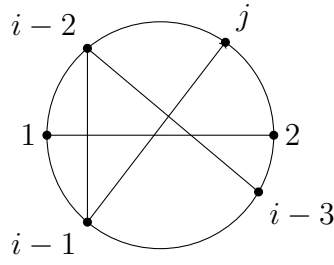


Figura 5.7: Caso B1.

Caso B2. $j < i - 1 < 2$. Consideriamo i sottocasi che dipendono dalla svolta in j .

Caso B21. In j la poligonale compie una svolta a destra. Rimpiazziamo l'indice $i - 1$ con l'indice j e incrementiamo j di 1.

Giustificazione. Con al più una traslazione debole in senso antiorario di $i - 3$ in un intorno di 2 possiamo assumere $2 < i - 3 < j$. Possiamo eliminare $i - 1$, la poligonale fino a j è stellata.

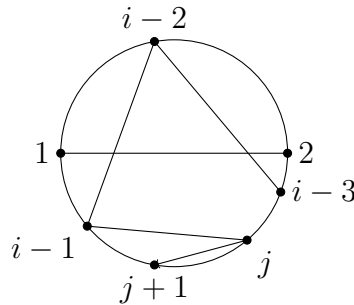


Figura 5.8: Caso B21.

Caso B22. In j la poligonale compie una svolta a sinistra. Consideriamo i due possibili casi che dipendono dalla posizione di $j + 1$.

Caso B221. $j < j + 1 < 1$. Consideriamo i casi che dipendono da $j + 2$.

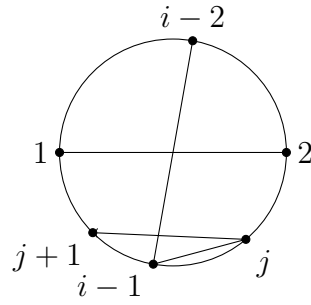


Figura 5.9: Caso B221.

Caso B2211. $j < j+2 < 2$. Dopo aver traslato j la poligonale è stellata. Incrementiamo sia i che j di 1.

Giustificazione. Possiamo traslare j in senso antiorario affinché $i-2 < j < 2$, ora la poligonale fino a j è stellata.

Caso B2212. $2 < j+2 < j$. Consideriamo i casi che dipendono da $j+3$.

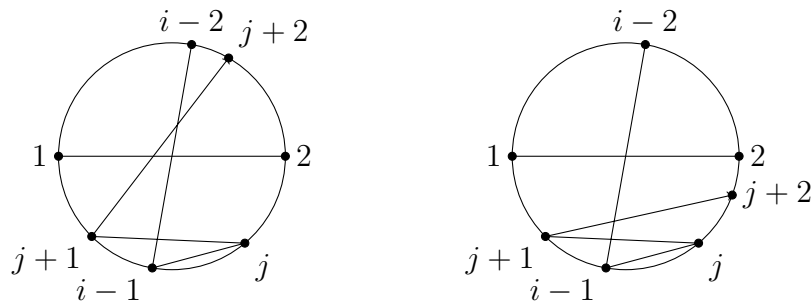


Figura 5.10: Casi B2211 e B2212.

Caso B22121. In $j+2$ la poligonale compie una svolta a sinistra. Incrementiamo j di 2.

Giustificazione. Possiamo eliminare prima l'indice $j+1$ e poi j .

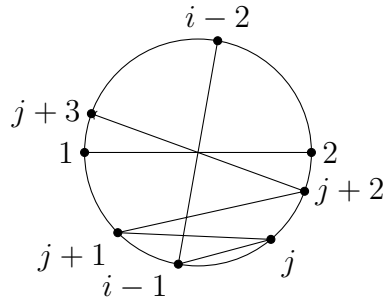


Figura 5.11: Caso B22121.

Caso B22122. In $j+2$ la poligonale compie una svolta a destra. Dopo qualche traslazione possiamo rimpiazzare l'indice $i-1$ con j . Incrementiamo i di 1 e j di 2.

Giustificazione. Dopo una traslazione debole in senso antiorario di $i-3$ in un intorno di 2, possiamo assumere $i-3 < j < i-1$. Possiamo allora traslare $j+1$ in senso orario affinché $i-2 < j+1 < 2$ ed eliminare $i-1$. La poligonale fino a $j+1$ è stellata.

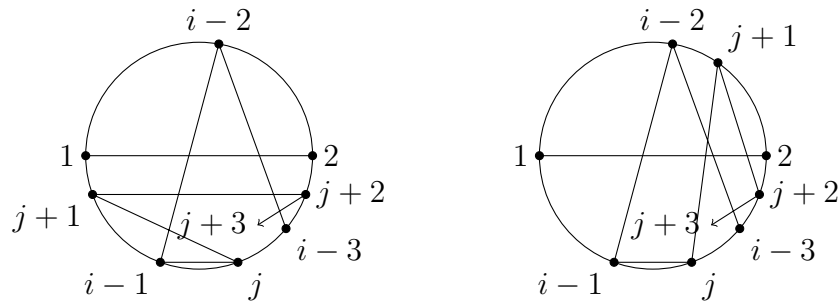


Figura 5.12: Caso B22122.

Caso B222. $1 < j+1 < j$. Dopo qualche traslazione, possiamo rimpiazzare l'indice $i-1$ con j ed incrementare j di 1.

Giustificazione. Effettuiamo una traslazione debole in senso antiorario di $i-2$ in un intorno di 1 in modo che $i-2 < j+1 < j$ come nella prima figura. Poi effettuiamo una traslazione debole in senso antiorario di $i-3$ in un intorno di 2 in modo che $i-3 < j < i-1$. Ora eliminiamo $i-1$, la poligonale fino a j è stellata.

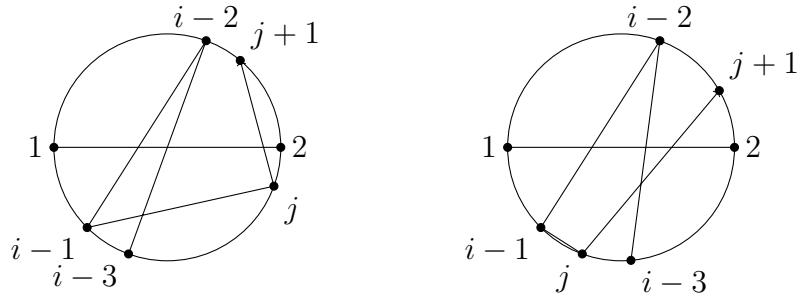
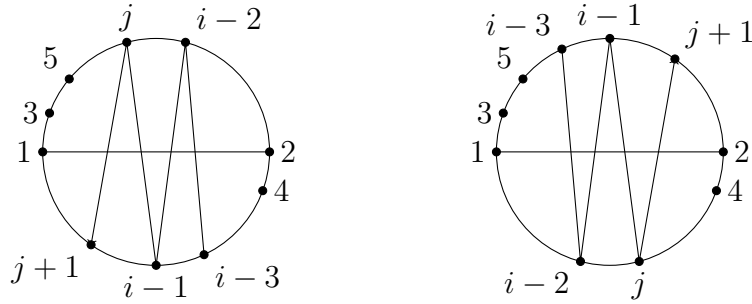


Figura 5.13: Caso B222.

Il caso per i pari è simmetrico; in particolare, poiché il caso rimane quello sinistrorso, gli ordini ciclici dei vertici già processati saranno gli stessi. Cambierà però che gli indici $i-1, i-3, \dots$ saranno dispari e quindi i vertici apparterranno all'arco di circonferenza sotto, e viceversa per $i-2, i-4, \dots$, i cui vertici apparterranno all'altro arco. Infine, cambierà anche il ruolo del primo e del secondo vertice, che verrà invertito.

Figura 5.14: Caso A1 per i dispari e analogo per i pari.

Abbiamo quindi dato un algoritmo che lavorando vertice per vertice trasforma qualsiasi poligonale chiusa con i vertici giacenti su una circonferenza nella n -stella del teorema.

Capitolo 6

L'implementazione in Python

6.1 Il progetto

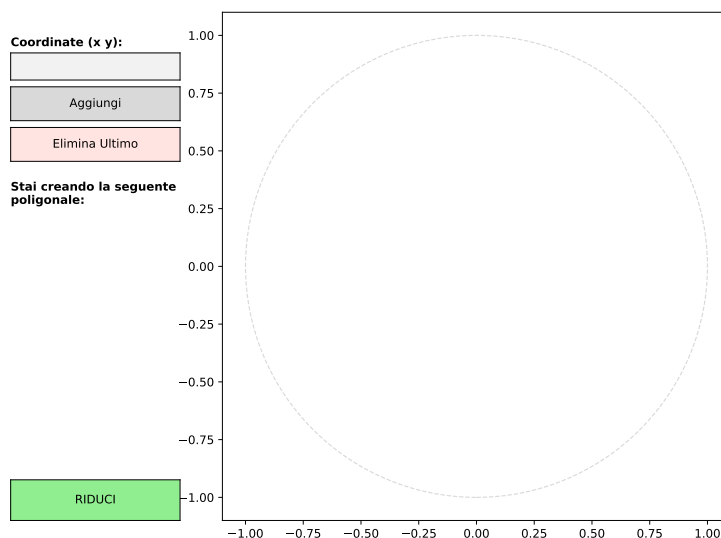
Il teorema presentato da Mehlhorn e Yap è di tipo costruttivo, ciò vuol dire che non si limita a dimostrare che un oggetto matematico esiste, ma fornisce anche un metodo per costruirlo. Nel nostro caso, partendo da una poligonale chiusa regolare, possiamo trasformarla in una n -stella tramite l'algoritmo del capitolo 4. L'idea naturale che può nascere da un teorema costruttivo è quella di implementarlo a livello informatico, ovvero sviluppare un codice che segue i passaggi di costruzione. In questo caso, l'obiettivo sarà quello di fornire una poligonale chiusa regolare ad un programma tramite una lista di vertici e trasformarla in una n -stella. I passaggi verranno anche rappresentati a livello visivo da un'interfaccia grafica. A questo scopo, il linguaggio di programmazione da noi scelto è Python. Python è un linguaggio di programmazione di alto livello, interpretato e orientato agli oggetti. Nello sviluppo del programma abbiamo seguito i principi del Test-Driven Development, una metodologia in cui la stesura dei test funzionali precede la scrittura del codice effettivo. Abbiamo utilizzato il framework `pytest` per strutturare e verificare i test che abbiamo raccolto in un file dedicato. L'interfaccia grafica interattiva è stata interamente sviluppata tramite la libreria `matplotlib`. Quest'ultima ci ha permesso non solo di tracciare la poligonale, ma di creare un vero e proprio visualizzatore dinamico che chiede i vertici di una poligonale, la riduce e mostra il processo di trasformazione passo dopo passo.

Per implementare l'algoritmo di Mehlhorn e Yap abbiamo creato gli oggetti `Vertex` e `Polygonal`. I lemmi e le condizioni che costituiscono l'ossatura del

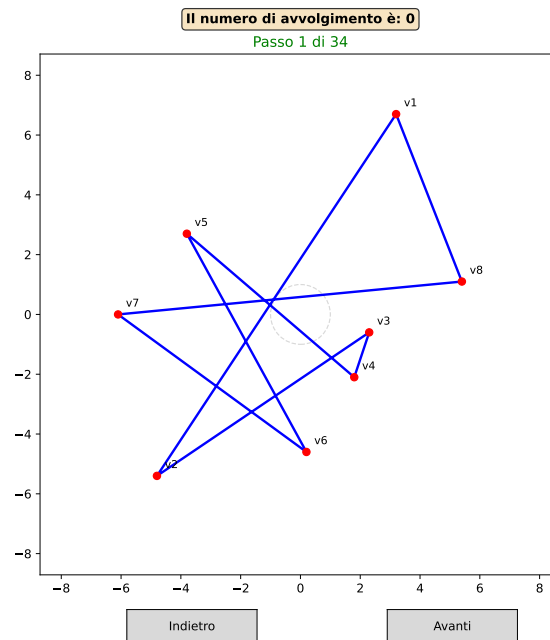
teorema costruttivo sono stati implementati come metodi di queste classi. Per concetti geometrici come la verifica degli ordini ciclici, il controllo della regolarità di una traslazione e le traslazioni deboli abbiamo tradotto le definizioni matematiche in metodi tramite codice rigoroso e funzionale. Un aspetto cruciale dello sviluppo ha riguardato l'estensione sistematica dei casi analizzati nell'articolo originale. Mentre il lavoro di Mehlhorn e Yap si concentra sulla descrizione dettagliata di 11 configurazioni fondamentali, l'implementazione informatica ha richiesto la scrittura esplicita di tutte le varianti simmetriche. In particolare, è stato necessario trattare autonomamente i casi relativi agli indici pari e le configurazioni che prevedono svolte a destra del secondo vertice, portando il totale a 44 casi distinti. Durante questa fase sono emerse complicazioni specifiche legate ai casi in cui il vertice da processare è il quarto, o l'ultimo. In tali circostanze, è stata necessaria una trattazione differente rispetto ai casi standard. Infine, abbiamo implementato poi tutti i casi dell'algoritmo e li abbiamo messi assieme con le relative verifiche nel metodo principale.

6.2 Il programma

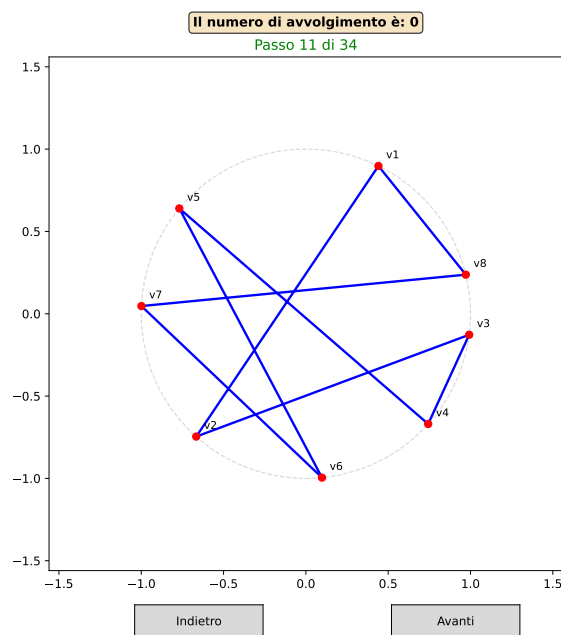
All'avvio del file `main.py` comparirà la seguente interfaccia, che chiede di inserire i vertici della poligonale. Una volta finito, si potrà premere il bottone "Riduci" per avviare il processo di riduzione a n -stella.



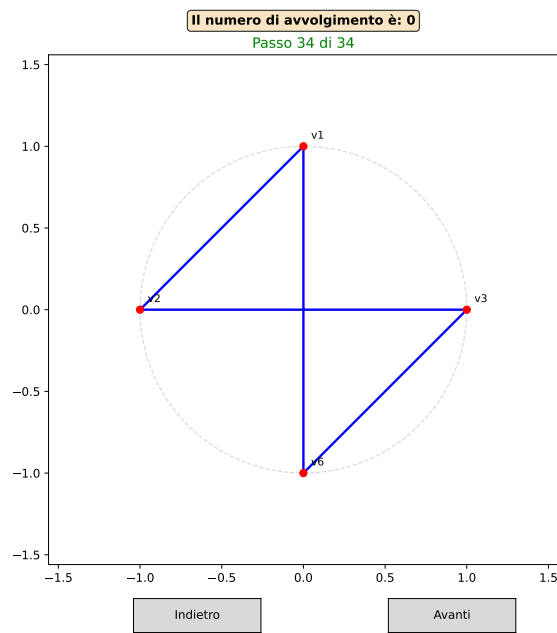
Dopo aver premuto il bottone "Riduci", compariranno la poligonale iniziale e il suo numero di avvolgimento.



Il metodo per ridurre la poligonale come nell'articolo di Mehlhorn e Yap porterà poi ad uno ad uno i vertici sulla circonferenza preservandone la regolarità.



Infine, applicando tutti i vari passaggi, ne effettuerà la riduzione.



A completamento di questa trattazione, nell'appendice verranno riportati e analizzati i frammenti più rilevanti del codice sorgente, offrendo uno sguardo diretto sulle strutture logiche e implementative alla base del programma.

Bibliografia

- [1] Kurt Mehlhorn and Chee-Keng Yap. Constructive Whitney-Graustein theorem: Or how to untangle closed planar curves. *SIAM Journal on Computing*, 20(4):603–621, 1991.
- [2] James R. Munkres. *Topology*. Prentice Hall, Upper Saddle River, NJ, 2nd edition, 2000.
- [3] Gert Vegter. Kink-free deformations of polygons. In *Proceedings of the fifth annual symposium on Computational geometry*, pages 61–68. ACM, 1989.
- [4] Hassler Whitney. On regular closed curves in the plane. *Compositio Mathematica*, 4:276–284, 1937.

Appendice: il codice

In questa appendice vengono presentati gli estratti più significativi del codice Python sviluppato per la realizzazione del progetto.

```
# Librerie importate
import math
import copy
import random
# Parametro di tolleranza scelto
TOLERANCE = 0.0005

def is_near(a : float, b : float) -> bool:
    """Verifica la vicinanza di a e b."""
    return math.fabs(a - b) < TOLERANCE
```

```
class Vertex:
    def __init__(self, name: str, x: float, y: float) -> None:
        """Inizializza un vertice con nome, coordinate (x, y)
        e calcola l'angolo rispetto all'origine."""
        # Nome del vertice
        self.name = name
        # Coordinate del vertice
        self.x = float(x)
        self.y = float(y)
        # Restituisce l'angolo fra 0 e 2*pi
        self.angle = math.atan2(self.y, self.x) % (2 * math.pi)

    def change_coordinates(self, x: float, y: float) -> None:
        """Sposta il vertice a nuove coordinate (x, y)
        e ricalcola l'angolo."""
        # Modifico le coordinate
        self.x = x
        self.y = y
        # Ricalcolo l'angolo
        self.angle = math.atan2(y, x) % (2 * math.pi)

    def change_angle(self, angle: float) -> None:
        """Sposta il vertice sulla circonferenza unitaria (R=1)
        in base al nuovo angolo fornito."""
        self.angle = angle
        self.x = math.cos(angle)
        self.y = math.sin(angle)
```

```

class Polygonal:
    def __init__(self, vertices: list[Vertex] = None) -> None:
        """Inizializza una poligonale come una lista di vertici.
        Se non viene fornita una lista di vertici
        inizializza una poligonale vuota."""
        if vertices is None:
            self.vertices = []
        else:
            self.vertices = vertices
        self.history = [vertices]

    def save_state(self) -> None:
        """Salva lo stato attuale della poligonale nella cronologia."""
        # Aggiungo lo stato attuale alla lista
        self.history.append(copy.deepcopy(self.vertices))

    def is_empty(self) -> bool:
        """Verifica se la poligonale è vuota."""
        return len(self.vertices) == 0

    def print_vertices(self) -> None:
        """Stampa nome e coordinate dei vertici della poligonale."""
        for vertex in self.vertices:
            print(f"{vertex.name}: ({vertex.x}, {vertex.y})")

    def get_v(self, i: int) -> Vertex:
        """Restituisce il vertice v_i, con i che parte da 1."""
        n = len(self.vertices)
        # Gestisce il caso poligonale vuota
        if n == 0:
            return None
        return self.vertices[(i - 1) % n]

    def add_vertex(self, V: Vertex) -> None:
        """Aggiunge un vertice alla poligonale."""
        self.vertices.append(V)

    def eliminate_vertex(self, i: int) -> None:
        """Elimina il vertice i-esimo se appartiene
        al segmento [v_im1, vip1]."""
        v_im1 = self.get_v(i - 1)
        v_i = self.get_v(i)
        vip1 = self.get_v(i + 1)
        # Controllo se i vettori sono allineati con il determinante
        det = (v_im1.x - v_i.x) * (vip1.y - v_i.y) - (v_im1.y - v_i.y) * (
            vip1.x - v_i.x
        )
        # Controllo se il punto è interno al segmento
        is_between_x = min(v_im1.x, vip1.x) <= v_i.x <= max(v_im1.x, vip1.x)
        is_between_y = min(v_im1.y, vip1.y) <= v_i.y <= max(v_im1.y, vip1.y)
        print(f"Ho eliminato il vertice {self.get_v(i).name}")
        if is_near(det, 0.0) and is_between_x and is_between_y:
            self.vertices.remove(v_i)
        else:
            raise ValueError("Il vertice non appartiene al segmento.")
        self.save_state()

```

```

def get_rotation_angle(self, i: int) -> float:
    """Calcola l'angolo di rotazione di v_i."""
    v_i = self.get_v(i)
    vip1 = self.get_v(i + 1)
    v_im1 = self.get_v(i - 1)
    # Calcolo dei vettori v_im1->v_i e v_i->vip1
    vec1_x = v_i.x - v_im1.x

```



```

vec1_y = v_i.y - v_im1.y
vec2_x = v_ip1.x - v_i.x
vec2_y = v_ip1.y - v_i.y
# Calcolo prodotto scalare e norme
dot_product = vec1_x * vec2_x + vec1_y * vec2_y
norm_1 = math.sqrt(vec1_x**2 + vec1_y**2)
norm_2 = math.sqrt(vec2_x**2 + vec2_y**2)
# Se due vertici sono identici, la norma è 0
if norm_1 == 0.0 or norm_2 == 0.0:
    return 0.0
# Calcolo il valore del coseno
cos_val = dot_product / (norm_1 * norm_2)
# Gestisco errori di approssimazione
cos_val = max(-1.0, min(1.0, cos_val))
abs_value_rot_vi = math.acos(cos_val)
det = vec1_x * vec2_y - vec1_y * vec2_x
rot_vi = math.copysign(abs_value_rot_vi, det)
return rot_vi

def get_winding_number(self) -> int:
    """Calcola l'indice di avvolgimento della poligonale."""
    sum_rotation_angles = 0
    # Calcola la somma degli angoli di rotazione
    # per tutti i vertici della poligonale
    for j in range(1, len(self.vertices) + 1):
        rot_vj = self.get_rotation_angle(j)
        sum_rotation_angles = sum_rotation_angles + rot_vj
    winding_number = sum_rotation_angles / (2 * math.pi)
    return round(winding_number)

```

```

def is_left_turn(self, i: int) -> bool:
    """Verifica se in v_i svolta a sinistra."""
    if self.get_rotation_angle(i) > 0:
        return True
    return False

```

```

def is_circle(self) -> bool:
    """Verifica se i vertici della poligonale appartengono
    ad una circonferenza."""
    n = len(self.vertices)
    if n < 3:
        return False # Almeno 3 punti per definire una circonferenza
    # Caso circonferenza unitaria per non avere errori con vertici vicini
    if all(is_near(v.x**2 + v.y**2, 1.0) for v in self.vertices):
        return True
    # Calcola circocentro e raggio usando i primi 3 vertici
    V1, V2, V3 = self.vertices[0], self.vertices[1], self.vertices[2]

    def calculate_circumcenter(V1: Vertex, V2: Vertex, V3: Vertex) -> Vertex:
        """Calcola il circocentro dei tre vertici V1, V2, V3."""
        # Calcola i punti medi dei segmenti V1-V2 e V2-V3
        M_12 = Vertex("Midpoint12", (V1.x + V2.x) / 2, (V1.y + V2.y) / 2)
        M_23 = Vertex("Midpoint23", (V2.x + V3.x) / 2, (V2.y + V3.y) / 2)
        # Calcola coefficiente delle rette perpendicolari a V1-V2 e V2-V3
        if is_near(V1.x, V2.x):
            # Il segmento è verticale, la perpendicolare è orizzontale
            m_perp_12 = 0.0
        elif is_near(V1.y, V2.y):
            # Il segmento è orizzontale, la perpendicolare è verticale
            m_perp_12 = float("inf")
        else:
            # Caso normale
            m_12 = (V1.y - V2.y) / (V1.x - V2.x)

```

```

        m_perp_12 = -1 / m_12
    if is_near(V2.x, V3.x):
        m_perp_23 = 0.0
    elif is_near(V2.y, V3.y):
        m_perp_23 = float("inf")
    else:
        m_23 = (V2.y - V3.y) / (V2.x - V3.x)
        m_perp_23 = -1 / m_23
    # Intersezione delle due rette perpendicolari
    if m_perp_12 == float("inf"):
        x = M_12.x
        y = m_perp_23 * (x - M_23.x) + M_23.y
    elif m_perp_23 == float("inf"):
        x = M_23.x
        y = m_perp_12 * (x - M_12.x) + M_12.y
    else:
        A = [[-m_perp_12, 1], [-m_perp_23, 1]]
        b = [M_12.y - m_perp_12 * M_12.x, M_23.y - m_perp_23 * M_23.x]
        # Risolvo il sistema lineare Ax = b
        det_A = A[0][0] * A[1][1] - A[0][1] * A[1][0]
        if det_A == 0:
            return None # Le rette sono parallele
        x = (b[0] * A[1][1] - b[1] * A[0][1]) / det_A
        y = (A[0][0] * b[1] - A[1][0] * b[0]) / det_A
    return Vertex("Circumcenter", x, y)

center = calculate_circumcenter(V1, V2, V3)
radius = math.sqrt((V1.x - center.x) ** 2 + (V1.y - center.y) ** 2)
for v in self.vertices[3:]:
    distance = math.sqrt((v.x - center.x) ** 2 + (v.y - center.y) ** 2)
    if not is_near(distance, radius):
        return False
return True

```

```

def is_translation_regular(self, i: int, U: Vertex) -> bool:
    """Verifica se la traslazione del vertice
    i-esimo in un punto è regolare."""
    V_im2 = self.get_v(i - 2)
    V_im1 = self.get_v(i - 1)
    V_i = self.get_v(i)
    V_ip1 = self.get_v(i + 1)
    V_ip2 = self.get_v(i + 2)

    # Funzione per verificare i coni
    def is_in_cone(o: Vertex, A: Vertex, B: Vertex, Q: Vertex, P: Vertex) -> bool:
        """Verifica se il punto P sta nel cono
        definito da A,O,B contenente Q."""
        # Calcola gli angoli rispetto all'origine O (da -pi a pi)
        angle_A = math.atan2(A.y - o.y, A.x - o.x)
        angle_B = math.atan2(B.y - o.y, B.x - o.x)
        angle_Q = math.atan2(Q.y - o.y, Q.x - o.x)
        angle_P = math.atan2(P.y - o.y, P.x - o.x)
        # Tiene OA come riferimento
        diff_B = (angle_B - angle_A) % (2 * math.pi)
        diff_Q = (angle_Q - angle_A) % (2 * math.pi)
        diff_P = (angle_P - angle_A) % (2 * math.pi)
        if diff_Q < diff_B < diff_P or diff_P < diff_B < diff_Q:
            return False
        else:
            return True

    # Cono relativo a v_i in v_im1
    # Punto opposto a v_ip1 sulla semiretta v_im1-vip1
    test_point_v_im1 = Vertex(
        "Test point 1", V_im1.x - (V_ip1.x - V_im1.x), V_im1.y - (V_ip1.y - V_im1.y)
    )

```

```

)
# P deve appartenere al cono contenente v_i
# formato da v_im1-test_point_v_im1
if is_in_cone(V_im1, test_point_v_im1, V_im2, V_i, U) is False:
    return False
# Cono relativo a v_i in v_ip1
# Punto opposto a v_im1 sulla semiretta v_ip1-vim1
test_point_v_ip1 = Vertex(
    "Test point 2", V_ip1.x - (V_im1.x - V_ip1.x), V_ip1.y - (V_im1.y - V_ip1.y)
)
# P deve appartenere al cono contenente v_i
# formato da v_im1-test_point_v_im1
if is_in_cone(V_ip1, test_point_v_ip1, V_ip2, V_i, U) is False:
    return False
return True

```

```

def move_to_midpoint(self, i: int) -> None:
    """Trasla il vertice i-esimo nel punto medio del segmento con
    estremi i vertici precedente e successivo."""
    V_im1 = self.get_v(i - 1)
    V_ip1 = self.get_v(i + 1)
    # Punto medio
    M = Vertex("M", (V_im1.x + V_ip1.x) / 2, (V_im1.y + V_ip1.y) / 2)
    if self.is_translation_regular(i, M) is True:
        self.get_v(i).change_coordinates(
            (V_im1.x + V_ip1.x) / 2, (V_im1.y + V_ip1.y) / 2
        )
    else:
        raise ValueError("Non si può fare con trasformazioni regolari.")
    self.save_state()

def move_and_eliminate(self, i: int) -> None:
    """Trasla il vertice i-esimo nel punto medio del segmento con
    estremi i vertici precedente e successivo e lo elimina."""
    self.move_to_midpoint(i)
    self.eliminate_vertex(i)
    print(f"Ho eliminato il vertice {self.get_v(i).name}")

```

```

def move_to_circle(self) -> None:
    """
    Centra la poligonale nell'origine e proietta i vertici sulla
    circonferenza unitaria."""
    # Centriamo la poligonale nell'origine
    self.center_polygonal()
    # Verifico se i vertici sono già su una circonferenza
    if self.is_circle() is True:
        self.get_unitary_radius()
        return None
    n = len(self.vertices)
    # Calcoliamo il raggio massimo
    r = max(math.hypot(v.x, v.y) for v in self.vertices)
    # Ciclo sui vertici (da 1 a n)
    for i in range(1, n + 1):
        v = self.get_v(i)
        # Coordinate del vertice proiettato sul cerchio
        new_x = r * math.cos(v.angle)
        new_y = r * math.sin(v.angle)
        new_vertex = Vertex(v.name, new_x, new_y)
        new_angle = new_vertex.angle
        success_a = False
        # Proviamo la proiezione diretta
        if self.is_translation_regular(i, new_vertex) is True:
            while True:

```

```

        if self.is_translation_regular(i, new_vertex) is False:
            break
        collision = False
        for z in self.vertices:
            if z.name == v.name:
                continue
            if is_near(z.x, new_vertex.x) and is_near(z.y, new_vertex.y):
                collision = True
                break
        if collision:
            new_angle += 10 * TOLERANCE
            new_x = r * math.cos(new_angle)
            new_y = r * math.sin(new_angle)
            continue # Ricontrolla tutti i vertici z con il nuovo angolo
        v.change_coordinates(new_x, new_y)
        self.save_state()
        success_a = True
        break
# Ricerca sui punti medi (se la proiezione diretta ha fallito)
if not success_a:

    def get_arc_midpoint(v1: Vertex, v2: Vertex) -> float:
        """Ritorna l'angolo del punto medio di un arco tra i vertici."""
        sin_sum = math.sin(v1.angle) + math.sin(v2.angle)
        cos_sum = math.cos(v1.angle) + math.cos(v2.angle)
        return math.atan2(sin_sum, cos_sum)

    neighbors = [
        self.get_v(i - 2),
        self.get_v(i - 1),
        self.get_v(i + 1),
        self.get_v(i + 2),
    ]
    candidates = []
    for idx_a in range(len(neighbors)):
        for idx_b in range(idx_a + 1, len(neighbors)):
            mid_angle = get_arc_midpoint(neighbors[idx_a], neighbors[idx_b])
            candidates.append(mid_angle)
    success_b = False
    # Proviamo i candidati uno ad uno
    for cand_angle in candidates:
        new_x_cand = r * math.cos(cand_angle)
        new_y_cand = r * math.sin(cand_angle)
        new_vertex = Vertex(v.name, new_x_cand, new_y_cand)
        new_angle = cand_angle # Parto dall'angolo del candidato
        while True:
            if self.is_translation_regular(i, new_vertex) is False:
                break
            collision_cand = False
            for z in self.vertices:
                if z.name == v.name:
                    continue
                if is_near(z.x, new_vertex.x) and is_near(
                    z.y, new_vertex.y
                ):
                    collision_cand = True
                    break
            if collision_cand:
                new_angle += 10 * TOLERANCE
                new_x = r * math.cos(new_angle)
                new_y = r * math.sin(new_angle)
                continue # Ricontrolla tutti i vertici
            success_b = True
            break
    if success_b:
        v.change_coordinates(new_x, new_y)
        self.save_state()

```

```

        break # Esco dal ciclo dei candidati
self.get_unitary_radius()

```

```

def is_clockwise(self, list_vertices: list[Vertex]) -> bool:
    """Verifica se i vertici sulla circonferenza sono
    ordinati in senso orario."""
    if self.is_circle() is False:
        raise ValueError("I vertici non appartengono ad una circonferenza.")
    cleaned_list = [list_vertices[0]]
    # Tolgo uno dei vertici consecutivi uguali
    for i in range(1, len(list_vertices)):
        v_current = list_vertices[i]
        v_prev = cleaned_list[-1]
        # Se il vertice corrente è diverso dall'ultimo inserito, lo aggiungo
        if v_current != v_prev and not is_near(v_current.angle, v_prev.angle):
            cleaned_list.append(v_current)
    # Se la lista ha più di un elemento, confronto il primo con l'ultimo
    if len(cleaned_list) > 1:
        first_v = cleaned_list[0]
        last_v = cleaned_list[-1]
        # Se l'ultimo è praticamente coincidente con il primo, lo elimino
        if last_v == first_v or is_near(last_v.angle, first_v.angle):
            cleaned_list.pop()
    list_vertices = cleaned_list
    n = len(list_vertices)
    if n <= 2:
        return True
    # Prendo l'angolo del primo vertice come riferimento
    base_angle = list_vertices[0].angle
    for j in range(1, n - 1):
        current_angle = list_vertices[j].angle
        # Calcolo la differenza angolare rispetto all'angolo di riferimento
        # Sarà crescente in senso antiorario
        current_angle_normalised = (current_angle - base_angle) % (2 * math.pi)
        next_angle = list_vertices[(j + 1) % n].angle
        next_angle_normalised = (next_angle - base_angle) % (2 * math.pi)
        # Calcoliamo la differenza fra angoli compresi fra [0, 2pi]
        diff = next_angle_normalised - current_angle_normalised
        # Gestione caso vertici coincidenti o vicini
        v_current = list_vertices[j]
        v_next = list_vertices[(j + 1) % n]
        if v_current == v_next or is_near(v_current.angle, v_next.angle):
            continue
        # Se diff è positiva percorrendo la circonferenza
        # a partire dal vertice base i due sono in senso antiorario
        if diff > 0 and not is_near(diff, 2 * math.pi):
            return False
        # Gestisco il caso in cui il vertice è vicino al vertice base
    return True

def is_counterclockwise(self, list_vertices: list[Vertex]) -> bool:
    """Verifica se i vertici sono in senso antiorario."""
    return self.is_clockwise(list_vertices[::-1])

```

```

def sort_vertices_clockwise(self, list_vertices: list[Vertex]) -> list[Vertex]:
    """Restituisce i vertici ordinati in senso orario
    a partire dal primo vertice della lista."""
    if self.is_circle() is False:
        raise ValueError("I vertici non appartengono ad una circonferenza.")
    if self.is_clockwise(list_vertices):
        return list_vertices
    else:
        # Calcola gli angoli rispetto

```

```

# all'angolo del primo vertice
base_angle = list_vertices[0].angle

def clockwise_distance(v):
    diff = (base_angle - v.angle) % (2 * math.pi)
    return diff

# Il primo rimarrà in posizione 0 perché la sua diff è 0
return sorted(list_vertices, key=clockwise_distance)

```

```

def get_equipaced_vertices(self) -> None:
    """Equidistanza i vertici della poligonale sulla circonferenza"""
    if self.is_circle() is False:
        raise ValueError("I vertici non appartengono ad una circonferenza.")
    n = len(self.vertices)
    sorted_vertices = self.sort_vertices_clockwise(self.vertices)
    base_angle = sorted_vertices[0].angle
    # Assegna ad ogni vertice un nuovo angolo equipaziato
    for j in range(n):
        # Sottraiamo per andare in senso orario
        new_angle = (base_angle - (2 * math.pi * j / n)) % (2 * math.pi)
        sorted_vertices[j].change_angle(new_angle)
    self.save_state()

def get_equipaced_vertices_fixed_12(self) -> None:
    """Equidistanza i vertici sul cerchio, tenendo fermi v1 e v2."""
    if self.is_circle() is False:
        raise ValueError("I vertici non appartengono ad una circonferenza.")

    n = len(self.vertices)
    if n <= 2:
        return # Niente da spostare
    # Troviamo i vertici fissi
    v1 = self.get_v(1)
    v2 = self.get_v(2)
    # Otteniamo l'ordine orario attuale di tutti i vertici sul cerchio
    sorted_vertices = self.sort_vertices_clockwise(self.vertices)
    # Estraiamo solo i vertici che si trovano nel percorso da v2 a v1
    idx_v2 = sorted_vertices.index(v2)
    vertices_to_move = []
    curr_idx = (idx_v2 + 1) % n
    while sorted_vertices[curr_idx].name != v1.name:
        vertices_to_move.append(sorted_vertices[curr_idx])
        curr_idx = (curr_idx + 1) % n
    # Calcoliamo lo spazio orario totale disponibile tra v1 e v2
    # di solito sarà pi greco
    total_gap = (v2.angle - v1.angle) % (2 * math.pi)
    # Dividiamo lo spazio
    num_steps = len(vertices_to_move) + 1
    step_size = total_gap / num_steps
    # Spalmiamo i vertici uno ad uno
    for i, v in enumerate(vertices_to_move):
        # Andiamo in senso orario
        new_angle = (v2.angle - step_size * (i + 1)) % (2 * math.pi)
        v.change_angle(new_angle)
    # Estraiamo solo i vertici che si trovano nel percorso da v1 a v2
    idx_v1 = sorted_vertices.index(v1)
    vertices_to_move = []
    curr_idx = (idx_v1 + 1) % n
    while sorted_vertices[curr_idx].name != v2.name:
        vertices_to_move.append(sorted_vertices[curr_idx])
        curr_idx = (curr_idx + 1) % n
    # Calcoliamo lo spazio orario totale disponibile tra v1 e v2
    # di solito sarà pi greco
    total_gap = (v1.angle - v2.angle) % (2 * math.pi)

```

```

# Dividiamo lo spazio
num_steps = len(vertices_to_move) + 1
step_size = total_gap / num_steps
# Spalmiamo i vertici uno ad uno
for i, v in enumerate(vertices_to_move):
    # Andiamo in senso orario
    new_angle = (v1.angle - step_size * (i + 1)) % (2 * math.pi)
    v.change_angle(new_angle)
self.save_state()

```

```

def rotate_vertex(self, i: int, delta_angle: float) -> None:
    """Ruota il vertice i rispetto all'origine spostandolo di
    'delta_angle'. Un delta positivo ruota in senso antiorario,
    negativo in senso orario."""
    # Calcoliamo il nuovo angolo sommandolo a quello attuale
    new_angle = (self.get_v(i).angle + delta_angle) % (2 * math.pi)
    self.get_v(i).change_angle(new_angle)
    self.save_state()

```

```

def weak_translation_clockwise(self, i: int, target_angle: float) -> None:
    """Trasla il vertice i-esimo lungo la circonferenza unitaria
    in senso orario fino all'angolo target_angle preservando ordine
    e regolarità traslando tutti i vertici collegati
    (vedi lemma 3.15)."""
    if self.is_circle() is False:
        raise ValueError("I vertici non appartengono ad una circonferenza.")
    # Vertice finale
    target_x = math.cos(target_angle)
    target_y = math.sin(target_angle)
    target = Vertex("Target", target_x, target_y)
    # Vertice iniziale
    v_i = self.get_v(i)

    # Chiamo A l'arco fra v_i e target
    def is_in_arc_A(v: Vertex) -> bool:
        """Verifica se il vertice appartiene
        all'arco orario A tra v_i e punto target."""
        # Se v è in senso orario rispetto a v_i e target,
        # allora è in A
        return self.is_clockwise([self.get_v(i), v, target])

    def get_active_vertices() -> list[Vertex]:
        """Ritorna la lista dei vertici collegati che vanno traslati."""

        def is_edge_active(j: int, k: int) -> bool:
            """Verifica se il lato con estremi v_j e v_k
            è attivo (ovvero almeno dei suoi estremi è in A)."""
            if k == j + 1 or k == j - 1:
                v1 = self.get_v(j)
                v2 = self.get_v(k)
                return is_in_arc_A(v1) or is_in_arc_A(v2)
            else:
                return False

        n = len(self.vertices)
        # Creo l'insieme degli indici attivi
        active_indices = {i % n}
        curr_idx = i
        # Poligoni attivi "in avanti"
        while True:
            next_idx = curr_idx + 1
            # Se il lato (corrente, successivo) è attivo e
            # non l'abbiamo già visitato
            if (

```

```

        is_edge_active(curr_idx, next_idx)
        and (next_idx % n) not in active_indices
    ):
        # Aggiungo vertice e passo al successivo
        active_indices.add(next_idx % n)
        curr_idx = next_idx
    else:
        break
# Poligonali attive "all'indietro"
curr_idx = i
while True:
    prev_idx = curr_idx - 1
    # Se il lato (precedente, corrente) è attivo
    # e non l'abbiamo già visitato
    if (
        is_edge_active(prev_idx, curr_idx)
        and (prev_idx % n) not in active_indices
    ):
        active_indices.add(prev_idx % n)
        curr_idx = prev_idx
    else:
        break
# Costruiamo la lista finale prendendo i vertici
list_active_vertices = [self.get_v(idx) for idx in active_indices]
return list_active_vertices

# Lista dei vertici attivi
active_vertices = get_active_vertices()
# Lista dei vertici attivi di A
active_vertices_in_A = [v for v in active_vertices if is_in_arc_A(v)]
# Lista dei vertici attivi di A da traslare
# in ordine (ovvero ordinati in senso antiorario a partire da v_i)
active_vertices_in_A.remove(v_i)
active_vertices_in_A.insert(0, v_i)
active_vertices_in_A = list(
    reversed(self.sort_vertices_clockwise(active_vertices_in_A))
)
# Calcolo il vertice successivo al vertice target
# Vertici della poligonale e target ordinati
# in senso orario a partire da v1
sorted_vertices = self.sort_vertices_clockwise(self.vertices + [target])
# Chiediamo in che posizione si trova il target
indice_target = sorted_vertices.index(target)
# Estraiamo il vertice successivo.
v_base = sorted_vertices[(indice_target + 1) % len(sorted_vertices)]
# Vertici da spostare
M = len(active_vertices_in_A)
# Angolo di distanza per traslare i vertici
diff_angle = (target_angle - v_base.angle) % (2 * math.pi)
prev_angle = v_base.angle
for idx, v in enumerate(active_vertices_in_A):
    k = idx + 1
    angle_v = (v_base.angle + diff_angle * k / M) % (2 * math.pi)
    # Controllo anti-sovrapposizione con altri vertici
    while True:
        for w in self.vertices:
            if w.name == v.name:
                continue
            if is_near(w.angle, angle_v):
                angle_v -= 10 * TOLERANCE
        break
    v_x = math.cos(angle_v)
    v_y = math.sin(angle_v)
    v.change_coordinates(v_x, v_y)
self.save_state()

```



```
def get_next_clockwise(self, i: int) -> Vertex:
    """Restituisce il vertice successivo all'i-esimo in senso orario."""
    sorted_vertices = self.sort_vertices_clockwise(self.vertices)
    start_vertex = self.get_v(i)
    # Trovo in che posizione si trova ora nella lista ordinata
    current_idx = sorted_vertices.index(start_vertex)
    # Calcolo l'indice successivo.
    next_idx = (current_idx + 1) % len(sorted_vertices)
    # Ritorno il vertice trovato
    return sorted_vertices[next_idx]
```

```
def permute_vertices_backward(self):
    """
    Ruota i vertici all'indietro di 1 posizione .
    Il vecchio v2 diventa il nuovo v1,..., e il vecchio v1 diventa l'ultimo."""
    # Estraggo il primo elemento e lo metto alla fine
    self.vertices.append(self.vertices.pop(0))
    print("Ho permutato ciclicamente i vertici indietro di 1.")
```

```
def is_polygonal_reduced(self):
    n = len(self.vertices)
    if n == 4:
        return is_near(self.get_winding_number(), 0)
    else:
        return is_near(math.fabs(self.get_winding_number()), (n - 1) / 2)
```

Di seguito riportiamo solo 4 esempi di tutte le casistiche implementate.

```
def case_A1(self, i: int) -> int:
    """Svolge le operazioni del caso A1 tornando l'indice da processare."""
    print("Caso A1")
    self.move_to_midpoint(i - 1)
    self.eliminate_vertex(i - 1)
    return i - 1

def case_B222(self, i: int) -> int:
    """Svolge le operazioni del caso B222 tornando l'indice da processare."""
    print("Caso B222")
    if self.is_left_turn(2) is True:
        # Passaggio 1
        if not self.is_clockwise(
            [
                self.get_v(i - 2),
                self.get_v(i + 1),
                self.get_v(i),
            ]
        ):
            angle = (
                self.get_next_counterclockwise(i + 1).angle
                + self.get_v(i + 1).angle
            ) / 2
            self.weak_translation_counterclockwise(i - 2, angle)
    if not self.is_clockwise(
        [
            self.get_v(i - 2),
            self.get_v(i + 1),
            self.get_v(i),
        ]
    ):
        raise ValueError("Controllo dopo traslazione non riuscito.")
```

```

# Passaggio 2
if not self.is_clockwise(
    [
        self.get_v(i - 3),
        self.get_v(i),
        self.get_v(i - 1),
    ]
):
    angle = (2 * math.pi + self.get_v(i).angle) / 2
    self.weak_translation_counterclockwise(i - 3, angle)
if not self.is_clockwise(
    [
        self.get_v(i - 3),
        self.get_v(i),
        self.get_v(i - 1),
    ]
):
    raise ValueError("Controllo dopo traslazione non riuscito.")
self.move_to_midpoint(i - 1)
self.eliminate_vertex(i - 1)
return i - 1
else:
    # Passaggio 1
    if not self.is_counterclockwise(
        [
            self.get_v(i - 2),
            self.get_v(i + 1),
            self.get_v(i),
        ]
    ):
        angle = (
            self.get_next_clockwise(i + 1).angle + self.get_v(i + 1).angle
        ) / 2
        self.weak_translation_clockwise(i - 2, angle)

    if not self.is_counterclockwise(
        [
            self.get_v(i - 2),
            self.get_v(i + 1),
            self.get_v(i),
        ]
    ):
        raise ValueError("Controllo dopo traslazione non riuscito")

    # Passaggio 2
    if not self.is_counterclockwise(
        [
            self.get_v(i - 3),
            self.get_v(i),
            self.get_v(i - 1),
        ]
    ):
        angle = (self.get_v(2).angle + self.get_v(i).angle) / 2
        self.weak_translation_clockwise(i - 3, angle)
    if not self.is_counterclockwise(
        [
            self.get_v(i - 3),
            self.get_v(i),
            self.get_v(i - 1),
        ]
    ):
        raise ValueError("Controllo dopo traslazione non riuscito")
    self.move_to_midpoint(i - 1)
    self.eliminate_vertex(i - 1)
    return i - 1

```

```

def case_C22(self, i: int) -> int:
    """Svolge le operazioni del caso C22 tornando l'indice da processare."""
    print("Caso C22")
    if self.is_left_turn(2) is True:
        # Primo passaggio
        vertices_to_check = [
            self.get_v(i - 4),
            self.get_v(i),
            self.get_v(i - 2),
        ]
        if not self.is_clockwise(vertices_to_check):
            angle_1 = (2 * math.pi + self.get_v(i).angle) / 2
            angle_2 = (math.pi + self.get_v(i).angle) / 2
            self.weak_translation_counterclockwise(i - 4, angle_1)
            self.weak_translation_clockwise(i - 2, angle_2)

        vertices_check = [
            self.get_v(i - 4),
            self.get_v(i),
            self.get_v(i - 2),
        ]
        if not self.is_clockwise(vertices_check) is True:
            raise ValueError("Primo passaggio non riuscito")

        # Secondo passaggio
        vertices_to_check_2 = [
            self.get_v(i - 4),
            self.get_v(i - 1),
            self.get_v(i),
        ]
        if not self.is_clockwise(vertices_to_check_2):
            angle_3 = (self.get_v(i - 4).angle + self.get_v(i).angle) / 2
            if (i - 4) == 2:
                angle_3 = (2 * math.pi + self.get_v(i).angle) / 2
            self.weak_translation_clockwise(i - 1, angle_3)

        if (
            not self.is_clockwise(
                [
                    self.get_v(i - 4),
                    self.get_v(i - 1),
                    self.get_v(i),
                ]
            )
            is True
        ):
            raise ValueError("Secondo passaggio non riuscito")
        self.move_to_midpoint(i - 2)
        self.eliminate_vertex(i - 2)
        return i - 1
    else:
        # Primo passaggio
        vertices_to_check = [
            self.get_v(i - 4),
            self.get_v(i),
            self.get_v(i - 2),
        ]
        if not self.is_counterclockwise(vertices_to_check):
            angle_1 = (self.get_v(2).angle + self.get_v(i).angle) / 2
            angle_2 = (math.pi + self.get_v(i).angle) / 2
            self.weak_translation_clockwise(i - 4, angle_1)
            self.weak_translation_counterclockwise(i - 2, angle_2)
        vertices_check = [
            self.get_v(i - 4),
            self.get_v(i),

```

```

        self.get_v(i - 2),
    ]
    if not self.is_counterclockwise(vertices_check) is True:
        raise ValueError("Primo passaggio non riuscito")
    # Secondo passaggio
    vertices_to_check_2 = [
        self.get_v(i - 4),
        self.get_v(i - 1),
        self.get_v(i),
    ]
    if not self.is_counterclockwise(vertices_to_check_2):
        angle_3 = (self.get_v(i - 4).angle + self.get_v(i).angle) / 2
        if (i - 4) == 2:
            angle_3 = (self.get_v(2).angle + self.get_v(i).angle) / 2
        self.weak_translation_counterclockwise(i - 1, angle_3)
    if (
        not self.is_counterclockwise(
            [
                self.get_v(i - 4),
                self.get_v(i - 1),
                self.get_v(i),
            ]
        )
        is True
    ):
        raise ValueError("Secondo passaggio non riuscito")
    self.move_to_midpoint(i - 2)
    self.eliminate_vertex(i - 2)
    return i - 1

def case_D22122(self, i: int) -> int:
    """Svolge le operazioni del caso D22122 tornando l'indice da processare."""
    print("Caso D22122")
    if self.is_left_turn(2) is True:
        # Caso limite
        if i == 4:
            if (
                self.is_clockwise(
                    [
                        self.get_v(2),
                        self.get_v(5),
                        self.get_v(4),
                    ]
                )
                is True
            ):
                self.move_to_midpoint(3)
                self.eliminate_vertex(3)

                self.move_to_midpoint(4)
                self.eliminate_vertex(4)
                return 4
            else:
                self.permute_vertices_backward()
                self.weak_translation_clockwise(1, math.pi)
                self.weak_translation_clockwise(2, 0)
                return i
        if not self.is_clockwise(
            [
                self.get_v(i - 3),
                self.get_v(i),
                self.get_v(i - 1),
            ]
        ):
            # Traslo i-3 in un intorno di 1
            angle = (self.get_v(1).angle + self.get_next_clockwise(1).angle) / 2
            self.weak_translation_counterclockwise(i - 3, angle)

```

```

        if not self.is_clockwise(
            [
                self.get_v(i - 3),
                self.get_v(i),
                self.get_v(i - 1),
            ]
        ):
            raise ValueError("Controllo dopo traslazione non riuscito")
        if not self.is_clockwise(
            [
                self.get_v(i - 2),
                self.get_v(i + 1),
                self.get_v(1),
            ]
        ):
            # Traslo j+1
            angle = (
                self.get_next_counterclockwise(1).angle + self.get_v(1).angle
            ) / 2
            self.weak_translation_clockwise(i + 1, angle)
        if not self.is_clockwise(
            [
                self.get_v(i - 2),
                self.get_v(i + 1),
                self.get_v(1),
            ]
        ):
            raise ValueError("Controllo dopo traslazione non riuscito")

        self.move_to_midpoint(i - 1)
        self.eliminate_vertex(i - 1)
        return i - 1
    else:
        # Caso limite
        if i == 4:
            if (
                self.is_counterclockwise(
                    [
                        self.get_v(2),
                        self.get_v(5),
                        self.get_v(4),
                    ]
                )
                is True
            ):
                self.move_to_midpoint(3)
                self.eliminate_vertex(3)

                self.move_to_midpoint(4)
                self.eliminate_vertex(4)
                return i
            else:
                self.permute_vertices_backward()
                self.weak_translation_counterclockwise(1, math.pi)
                self.weak_translation_counterclockwise(2, 0)
                return i
        if not self.is_counterclockwise(
            [
                self.get_v(i - 3),
                self.get_v(i),
                self.get_v(i - 1),
            ]
        ):
            # Traslo i-3 in un intorno di 1
            angle = (
                self.get_v(1).angle + self.get_next_counterclockwise(1).angle
            ) / 2

```

```

        self.weak_translation_clockwise(i - 3, angle)
    if not self.is_counterclockwise(
        [
            self.get_v(i - 3),
            self.get_v(i),
            self.get_v(i - 1),
        ]
    ):
        raise ValueError("Controllo dopo traslazione non riuscito")
    if not self.is_counterclockwise(
        [
            self.get_v(i - 2),
            self.get_v(i + 1),
            self.get_v(1),
        ]
    ):
        # Traslo j+1
        angle = (self.get_next_clockwise(1).angle + self.get_v(1).angle) / 2
        self.weak_translation_counterclockwise(i + 1, angle)
    if not self.is_counterclockwise(
        [
            self.get_v(i - 2),
            self.get_v(i + 1),
            self.get_v(1),
        ]
    ):
        raise ValueError("Controllo dopo traslazione non riuscito")
    self.move_to_midpoint(i - 1)
    self.eliminate_vertex(i - 1)
    return i - 1

```

```

def reduce_polygonal(self):
    """Esegue l'algoritmo di riduzione di Mehlhorn-Yap sulla poligonale."""
    print("I vertici sono della poligonale da ridurre sono:")
    self.print_vertices
    self.save_state()
    n = len(self.vertices)
    # Casi banali
    if n == 1:
        print("La poligonale è un punto.")
    if n == 2:
        print("La poligonale è un segmento.")
    if n == 3:
        print("La poligonale è un triangolo.")
    # Elimino i vertici appartenenti ai lati
    for i in range(3, n + 1):
        if is_near(self.get_rotation_angle(i), 0):
            self.eliminate_vertex(i)
            print(f"Eliminato vertice {i} perché appartiene già ad un lato.")
    if self.is_polygonal_reduced():
        return None
    if not self.is_circle():
        self.move_to_circle()
        if not self.is_circle():
            raise ValueError("La curva poligonale non è inscritta in un cerchio.")
    # Porto la curva poligonale su circonferenza di raggio 1
    if any(not is_near(math.hypot(v.x, v.y), 1.0) for v in self.vertices):
        self.get_unitary_radius()
    # Inizio l'algoritmo dal quarto vertice
    i = 4
    while self.is_polygonal_reduced() is False:
        # Rimetto i vertici di riferimento apposto ogni volta
        # Porto il vertice v(1) in (-1,0)
        if not is_near(self.get_v(1).x, -1) or not is_near(self.get_v(1).y, 0):
            if self.get_v(1).angle < math.pi:

```

```

        self.weak_translation_counterclockwise(1, math.pi)
    else:
        self.weak_translation_clockwise(1, math.pi)
    print(
        f"Ho traslato il vertice di rif. {self.get_v(1).name} in (-1,0)."
    )
    # Porto il vertice v(2) in (1,0)
    if not is_near(self.get_v(2).x, 1) or not is_near(self.get_v(2).y, 0):
        if self.get_v(2).angle < math.pi:
            self.weak_translation_clockwise(2, 0)
        else:
            self.weak_translation_counterclockwise(2, 0)
    print(
        f"Ho traslato il vertice di rif. {self.get_v(2).name} in (1,0)."
    )
    print(f"Sto processando il vertice {self.get_v(i).name} con indice {i}.")
    # Caso svolta a destra in 2
    if self.is_right_turn(2):
        # Caso i dispari
        if i % 2 == 1:
            # Caso A
            if self.is_left_turn(i - 1):
                # Caso A1
                if self.is_right_turn(i):
                    i = self.case_A1(i)
                    continue
                # Caso A2
            else:
                # Caso A21
                if self.is_counterclockwise(
                    [
                        self.get_v(i),
                        self.get_v(i + 1),
                        self.get_v(2),
                    ]
                ):
                    i = self.case_A21(i)
                    continue
                # Caso A22
                if self.is_counterclockwise(
                    [
                        self.get_v(i),
                        self.get_v(2),
                        self.get_v(i + 1),
                        self.get_v(1),
                    ]
                ):
                    i = self.case_A22(i)
                # Caso A23
            else:
                if not self.is_counterclockwise(
                    [
                        self.get_v(i),
                        self.get_v(1),
                        self.get_v(2),
                        self.get_v(i + 1),
                    ]
                ):
                    raise ValueError(
                        "Controllo non riuscito."
                    )
                # Caso A231
                if (
                    self.is_counterclockwise(
                        [
                            self.get_v(i),
                            self.get_v(1),

```

```

        self.get_v(i + 2),
    ]
    )
    and (i + 2) % len(self.vertices) != 1
):
    i = self.case_A231(i)
    continue
# Caso A232
else:
    if not self.is_counterclockwise(
        [
            self.get_v(i),
            self.get_v(i + 2),
            self.get_v(1),
        ]
    ):
        raise ValueError(
            "Controllo non riuscito."
        )
    # Caso A2321
    if self.is_left_turn(i + 2):
        i = self.case_A2321(i)
    # Caso A2322
    else:
        i = self.case_A2322(i)
# Caso B
else:
    # Caso B1
    if self.is_counterclockwise(
        [
            self.get_v(i),
            self.get_v(2),
            self.get_v(i - 2),
        ]
    ):
        i = self.case_B1(i)
        continue
    # Caso B2
    else:
        if not self.is_counterclockwise(
            [
                self.get_v(i),
                self.get_v(i - 1),
                self.get_v(2),
            ]
        ):
            raise ValueError("Controllo non riuscito.")
        # Caso B21
        if self.is_left_turn(i):
            i = self.case_B21(i)
            continue
        # Caso B22
        else:
            # Caso B221
            if self.is_counterclockwise(
                [
                    self.get_v(i),
                    self.get_v(i + 1),
                    self.get_v(1),
                ]
            ):
                # Caso B2211
                if (
                    self.is_counterclockwise(
                        [
                            self.get_v(i),
                            self.get_v(i + 2),

```



```

        self.get_v(2),
    ]
    )
    and not (i + 2) % len(self.vertices) == 2
):
    i = self.case_B2211(i)
    continue
# Caso B2212
else:
    if not self.is_counterclockwise(
        [
            self.get_v(2),
            self.get_v(i + 2),
            self.get_v(i),
        ]
    ):
        raise ValueError("Non entra in nessun caso.")
# Caso B22121
if self.is_right_turn(i + 2):
    i = self.case_B22121(i)
    continue
# Caso B22122
else:
    i = self.case_B22122(i)
    continue
# caso B222
else:
    if not self.is_counterclockwise(
        [
            self.get_v(1),
            self.get_v(i + 1),
            self.get_v(i),
        ]
    ):
        raise ValueError("Non entra in nessun caso.")
    i = self.case_B222(i)
    continue

# Caso i pari
elif i % 2 == 0:
    # Caso C
    if self.is_left_turn(i - 1):
        # Caso C1
        if self.is_right_turn(i):
            i = self.case_C1(i)
            continue
        # Caso C2
    else:
        # Caso limite j = 4
        if i == 4:
            if self.is_counterclockwise(
                [self.get_v(4), self.get_v(5), self.get_v(2)]
            ):
                self.move_to_midpoint(3)
                self.eliminate_vertex(3)
                i -= 1
                continue
            # Permuto gli indici così la poligonale è stellata per 4
        else:
            self.permute_vertices_backward()
            self.weak_translation_counterclockwise(1, math.pi)
            self.weak_translation_counterclockwise(2, 0)
            continue
        # Caso C21
        if self.is_counterclockwise(
            [
                self.get_v(i),

```

```

        self.get_v(i + 1),
        self.get_v(1),
    ]
    ):
        i = self.case_C21(i)
        continue
    # ... [Codice omissso per brevità] ...
    # Caso D
    else:
        # ... [Codice omissso per brevità] ...
    # Caso svolta a sinistra in 2
    if self.is_left_turn(2):
        # ... [Codice omissso per brevità] ...
    self.get_equispaced_vertices()

```

```

def generate_random_polygonal(number_vertices: int) -> Polygonal:
    """Genera una poligonale casuale con number_vertices vertici
    sulla circonferenza unitaria."""
    vertices = []
    for i in range(1, number_vertices + 1):
        # Genera coordinate casuali tra -1 e 1
        x = random.uniform(-1, 1)
        y = math.sqrt(1 - x**2) * random.choice([1, -1])
        vertices.append(Vertex(f"v{i}", x, y))

    return Polygonal(vertices)

```